

NUBOM

Ni Un Bocado Menos

Documento de Especificación Técnica y Arquitectura

Armando Luna Juárez

8 de octubre de 2026
Proyecto final · Diplomado Java

Contenido

2. Resumen ejecutivo

3. Requerimientos funcionales

- 3.1 Actores
- 3.2 Casos de uso
- 3.3 Lista de requerimientos
- 3.4 Operaciones CRUD
- 3.5 Reglas de negocio
- 3.6 Relaciones entre entidades

4. Requerimientos no funcionales

5. Arquitectura del sistema

- 5.1 Vista general
- 5.2 Tecnologías utilizadas
- 5.3 Diagrama de arquitectura
- 5.4 Estructura de paquetes
- 5.5 Flujo de autenticación (JWT + Refresh Token)

6. Modelo de datos

- 6.1 Diagrama ER
- 6.2 Descripción de tablas

7. Diseño del sistema

- 7.1 Diagramas UML
- 7.2 DTOs
- 7.3 Validaciones
- 7.4 Servicios y repositorios

8. Seguridad

- 8.1 Configuración de Spring Security
- 8.2 Filtros personalizados
- 8.3 JWT y Refresh Token
- 8.4 CSRF
- 8.5 Matriz de autorización

9. API REST

- 9.1 Documentación de endpoints
- 9.2 Consumo de API externa

10. Configuración

11. Evidencia de funcionamiento

12. Código fuente

13. Conclusiones

2. Resumen ejecutivo

2.1 Descripción general

NUBOM (Ni Un Bocado Menos) es una aplicación web para llevar el control de los alimentos que hay en casa, tanto en el refrigerador como en la alacena. En ella se registran los productos con su fecha de caducidad, se arma la lista de compras y se guardan recetas que también se pueden compartir. Todos los integrantes de una familia ven y actualizan el mismo inventario. La idea de fondo es desperdiciar menos comida, y para eso hace falta saber qué hay, qué está por vencer y qué falta comprar.

Lo fui construyendo a lo largo del diplomado sobre un solo proyecto de Spring Boot, agregando en cada módulo lo que se iba viendo. Tiene una interfaz web hecha con Spring Web MVC y Thymeleaf, que es la que usan las personas, y una API REST bajo `/api/v1` pensada para otros clientes. Las dos usan los mismos servicios y la misma base de datos. En esta entrega ambas quedaron protegidas con Spring Security. La web trabaja con formulario de inicio de sesión, sesión y protección CSRF, mientras que la API usa tokens JWT de corta duración y refresh tokens que se renuevan en cada uso.

2.2 Objetivo

El objetivo es desarrollar una aplicación en capas con Java 17, Spring e Hibernate que resuelva la administración compartida del inventario de alimentos de un hogar y que, de paso, ponga en práctica en un caso real lo visto en el diplomado: inyección de dependencias, persistencia con Spring Data JPA, desarrollo web con Spring Web MVC, servicios REST con Spring Boot y seguridad con Spring Security.

2.3 Alcance

El sistema cubre el inventario de alimentos del hogar, el seguimiento de caducidades, la lista de compras, las recetas (con la opción de compartirlas y calificarlas), la colaboración entre los integrantes de un mismo hogar y algunos reportes. Está dividido en seis módulos:

Módulo	Descripción
Usuarios y seguridad	Registro, inicio de sesión, perfil, cambio de contraseña y baja de cuenta. Autenticación y autorización con Spring Security para la web y para la API.
Inventario	Altas, bajas, cambios y consultas de productos en refrigerador y alacena, con vista gráfica del nivel de ocupación y estilos visuales configurables. Incluye la lista de compras del hogar.
Caducidad y notificaciones	Fecha de caducidad por producto, semáforo de estado y avisos automáticos diarios dentro de la aplicación y por correo electrónico.
Recetas	Altas, bajas, cambios y consultas de recetas propias con ingredientes principales y complementarios; compartición con la comunidad previa moderación, y calificación de 1 a 5.
Hogar compartido	Creación y administración del hogar, incorporación de integrantes y perfiles Propietario y Familiar.
Reportes	Productos próximos a caducar, productos por categoría, recetas mejor calificadas y nivel de ocupación.

Cambios de alcance respecto a la versión 1.0

- Agregué la lista de compras del hogar, la API REST completa, la consulta a Open Food Facts para autocompletar productos y el perfil de administrador, que modera las recetas y mantiene los catálogos.
- El «sistema de verificación sobre lo que suben los usuarios» de la primera versión quedó como un flujo de moderación. Una receta compartida no la ve nadie más hasta que un administrador la aprueba.

- Deje para después el motor que sugiere recetas según los ingredientes disponibles. La base de datos ya distingue entre ingredientes principales y complementarios, pero el cálculo todavía no está hecho.
- Siguen como mejoras futuras la generación de recetas con inteligencia artificial, la categorización automática y el escaneo de productos o tickets con la cámara.

3. Requerimientos funcionales

3.1 Actores

Actor	Descripción	Representación en el sistema
Visitante	Persona sin sesión. Solo puede registrarse o iniciar sesión.	Petición anónima
Familiar	Usuario autenticado que pertenece a un hogar. Consulta y actualiza el inventario y la lista de compras, administra sus recetas, califica y consulta reportes.	ROLE_USUARIO + rol FAMILIAR en usuario_hogar
Propietario	Familiar que creó el hogar. Además administra el hogar, sus integrantes y la configuración de los inventarios.	ROLE_USUARIO + rol PROPIETARIO en usuario_hogar
Administrador	Responsable de la plataforma. Modera las recetas compartidas y mantiene catálogos y usuarios.	ROLE_ADMIN (columna usuario.rol_sistema)
Tarea programada	Proceso interno que revisa caducidades una vez al día y depura tokens vencidos.	@Scheduled
Open Food Facts	Servicio externo que devuelve los datos de un producto a partir de su código de barras.	Cliente RestTemplate

3.2 Casos de uso

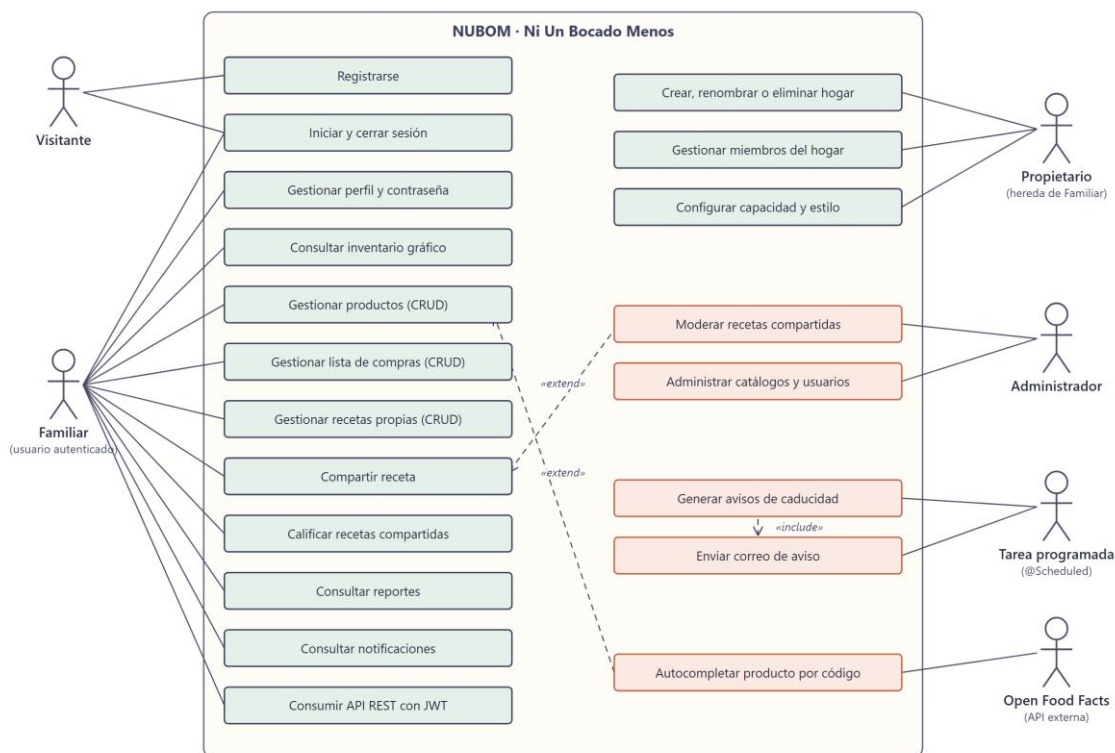


Figura 1. Diagrama de casos de uso. En naranja aparecen los casos que solo realiza el administrador y los que ejecuta el propio sistema.

Casos de uso detallados

Elemento	CU-05: Registrar producto en el inventario
Actor	Familiar o Propietario
Precondición	Sesión iniciada y pertenencia activa a un hogar.
Flujo principal	1) El actor elige refrigerador o alacena y abre el formulario de alta. 2) Opcionalmente captura el código de barras y el sistema propone nombre y categoría (CU-15). 3) Captura nombre, cantidad, unidad, categoría y fecha de caducidad. 4) El sistema valida los datos y la capacidad disponible. 5) Guarda el producto y actualiza la vista gráfica y el nivel de ocupación.
Flujos alternos	4a) Datos inválidos: se muestra el formulario con el mensaje por campo. 4b) Inventario lleno: se rechaza el alta e informa la capacidad máxima. 2a) Open Food Facts no responde o no conoce el código: el alta continúa con captura manual.
Postcondición	El producto queda activo en el inventario y entra al seguimiento de caducidad.

Elemento	CU-08: Compartir receta y CU-12: Moderar receta
Actores	Familiar (autor) y Administrador
Precondición	La receta existe, pertenece al autor y tiene al menos un ingrediente principal.
Flujo principal	1) El autor solicita compartir su receta; el estado pasa de PRIVADA a PENDIENTE . 2) El administrador revisa la bandeja de moderación. 3) Aprueba la receta: el estado pasa a APROBADA y aparece en el recetario de la comunidad. 4) Otros usuarios pueden calificarla.
Flujos alternos	3a) El administrador la rechaza: el estado pasa a RECHAZADA y solo el autor la ve. 1a) El autor edita una receta aprobada: vuelve a PENDIENTE .
Postcondición	Solo las recetas aprobadas son visibles y calificables por terceros.

3.3 Lista de requerimientos funcionales

ID	Módulo	Requerimiento
RF-01	Usuarios y seguridad	Registro de usuario con nombre, correo electrónico único y contraseña de al menos 8 caracteres con confirmación.
RF-02		Inicio y cierre de sesión en la interfaz web mediante formulario.
RF-03		Obtención, renovación y revocación de tokens para consumir la API REST.
RF-04		Consulta y edición del perfil, y cambio de contraseña verificando la contraseña actual.
RF-05		Baja lógica de la cuenta. No se permite mientras el usuario sea propietario de un hogar.
RF-06		Control de acceso por perfil: Familiar, Propietario y Administrador.
RF-07	Hogar compartido	Creación de un hogar; quien lo crea queda como Propietario y se generan automáticamente su refrigerador y su alacena.
RF-08		Renombrado y eliminación del hogar por el Propietario.
RF-09		Incorporación de integrantes por correo electrónico con perfil Familiar, consulta de miembros y retiro de integrantes.
RF-10		Todos los integrantes consultan y actualizan el mismo inventario y la misma lista de compras.
RF-11	Inventario	Alta, baja, cambio y consulta de productos, diferenciando refrigerador y alacena.
RF-12		Vista gráfica e interactiva de cada espacio de almacenamiento con sus productos y su nivel de ocupación.
RF-13		Configuración de la capacidad de cada inventario (20, 40 u 80 objetos) y de su estilo visual.
RF-14		Alta, baja, cambio y consulta de la lista de compras, y marcado de artículos como comprados.

ID	Módulo	Requerimiento
RF-15		Autocompletado de nombre y categoría de un producto a partir de su código de barras.
RF-16	Caducidad y notificaciones	Registro de la fecha de caducidad de cada producto y semáforo de estado: vigente, por vencer (3 días o menos) y vencido.
RF-17		Generación automática diaria de avisos para los integrantes del hogar cuando un producto está por vencer.
RF-18		Envío de un correo resumen con los productos por vencer.
RF-19		Consulta de avisos y marcado como leídos.
RF-20	Recetas	Alta, baja, cambio y consulta de recetas propias con nombre, descripción, pasos, tiempo e ingredientes (cantidad, unidad y distinción entre principal y complementario).
RF-21		Solicitud para compartir una receta con la comunidad.
RF-22		Moderación de recetas compartidas por el Administrador (aprobar o rechazar).
RF-23		Consulta paginada del recetario de la comunidad con búsqueda por nombre.
RF-24		Calificación de 1 a 5 de recetas ajenas aprobadas; una calificación por usuario y receta, modificable.
RF-25	Reportes	Reporte de productos próximos a caducar.
RF-26		Reporte de productos por categoría.
RF-27		Reporte de recetas mejor calificadas.
RF-28		Reporte de nivel de ocupación del refrigerador y de la alacena.
RF-29	Administración	Mantenimiento de catálogos (roles de hogar y tipos de inventario) y de usuarios por el Administrador a través de la API.

3.4 Operaciones CRUD

La plantilla pide al menos cinco CRUD. El sistema tiene nueve, repartidos entre la interfaz web y la API REST como se muestra en la tabla.

#	Entidad	Crear	Leer	Actualizar	Eliminar	Web	API
1	Usuario	Registro	Perfil / listado	Perfil y contraseña	Baja lógica	Sí	Sí
2	Hogar	Crear hogar	Resumen y miembros	Renombrar	Eliminar en cascada	Sí	Sí
3	Inventario	Automático / API	Por hogar	Capacidad y estilo	Eliminar	Parcial	Sí
4	Producto	Alta	Por inventario, paginado	Edición	Baja	Sí	Sí
5	Lista de compras	Agregar	Por hogar	Editar / comprado	Eliminar	Sí	Sí
6	Receta (con ingredientes)	Alta	Propias / comunidad	Edición	Eliminar en cascada	Sí	Sí
7	Calificación de receta	Calificar	Promedio y propias	Cambiar puntuación	Retirar	Sí	Sí
8	Tipo de inventario (catálogo)	Alta	Listado	Edición	Eliminar si no está en uso	No	Sí
9	Rol de hogar (catálogo)	Alta	Listado	Edición	Eliminar si no está en uso	No	Sí

3.5 Reglas de negocio

ID	Regla
RN-01	Al crear un hogar se generan dos inventarios: Refrigerador con capacidad de 20 objetos y Alacena con capacidad de 40.
RN-02	La capacidad de un inventario solo admite 20, 40 u 80 objetos y no puede reducirse por debajo del número de productos activos.
RN-03	No se puede agregar un producto a un inventario que alcanzó su capacidad máxima.
RN-04	Un usuario solo opera sobre los inventarios, productos y lista de compras del hogar al que pertenece.
RN-05	Solo el Propietario renombra o elimina el hogar, cambia capacidades y estilos, y gestiona integrantes.
RN-06	Un Propietario no puede dar de baja su cuenta mientras conserve ese rol en un hogar.
RN-07	El correo electrónico es único y se almacena normalizado (sin espacios y en minúsculas).
RN-08	Un producto está «por vencer» cuando faltan 3 días o menos para su caducidad. Se genera como máximo un aviso por producto, usuario y día.
RN-09	Una receta requiere al menos un ingrediente principal. Solo su autor puede editarla o eliminarla.
RN-10	Una receta es visible para terceros únicamente en estado APROBADA . Editar una receta aprobada la regresa a PENDIENTE .
RN-11	Un usuario no puede calificar sus propias recetas y solo tiene una calificación por receta.
RN-12	No se elimina un tipo de inventario ni un rol que esté en uso; la API responde 409 Conflict.

3.6 Relaciones entre entidades

Las relaciones están hechas con anotaciones de JPA, así que es Hibernate quien genera las llaves foráneas. En las bidireccionales el lado «uno» lleva `mappedBy` y el lado «muchos» usa carga perezosa. Para no caer en ciclos al convertir a JSON, las entidades no se devuelven tal cual y la API siempre responde con DTOs.

Relación	Tipo	Implementación
hogar – inventario	1:N bidireccional	<code>@OneToMany(mappedBy, cascade = ALL, orphanRemoval = true) / @ManyToOne(LAZY)</code>
inventario – producto	1:N bidireccional	Igual que la anterior: al eliminar un inventario se eliminan sus productos.
hogar – lista_compra	1:N bidireccional	Cascada completa desde el hogar.
inventario_tipo – inventario	1:N bidireccional	Sin cascada: no se permite eliminar un tipo en uso.
receta – receta_ingrediente	1:N bidireccional	Cascada completa: los ingredientes se guardan y eliminan con la receta.
receta – receta_calificacion	1:N bidireccional	Cascada de eliminación; restricción única (receta, usuario).
usuario – hogar	N:M con atributos	Entidad de unión <code>usuario_hogar</code> con tres <code>@ManyToOne</code> (usuario, hogar, rol), fecha de unión y bandera de activo.
usuario – receta, refresh_token, notificacion	1:N unidireccional	<code>@ManyToOne(LAZY)</code> desde el lado «muchos».

4. Requerimientos no funcionales

4.1 Seguridad

En esta versión el trabajo fuerte estuvo en la seguridad. La tabla resume lo que quedó funcionando y el detalle técnico se explica en la sección 8.

ID	Requerimiento	Funcionalidad alcanzada
RNF-S01	Autenticación web	Inicio de sesión por formulario propio con Spring Security; usuarios y roles recuperados de la base de datos mediante <code>UserDetailsService</code> .
RNF-S02	Autenticación de la API	JWT firmado con HS256 y vigencia de 15 minutos, validado en cada petición por un filtro propio. La API no crea sesión.
RNF-S03	Renovación de sesión de la API	Refresh token opaco de 7 días, guardado como hash SHA-256, de un solo uso (rotación) y revocable.
RNF-S04	Almacenamiento de contraseñas	Hash BCrypt. La contraseña nunca se devuelve en respuestas ni se escribe en bitácoras.
RNF-S05	Autorización por rol	Reglas por URL en cada cadena de filtros y <code>@PreAuthorize</code> en servicios para operaciones del Propietario y del Administrador.
RNF-S06	Aislamiento entre hogares	Cada operación verifica que el usuario autenticado pertenezca al hogar del recurso; de lo contrario la respuesta es 403.
RNF-S07	Protección CSRF	Activa en todos los formularios de la interfaz web; deshabilitada en la API por ser sin estado.
RNF-S08	Control de sesiones	Regeneración del identificador de sesión al autenticarse, una sesión activa por usuario, expiración a los 30 minutos y cookie <code>HttpOnly</code> con <code>SameSite=Lax</code> .
RNF-S09	Errores de seguridad uniformes	La API responde 401 y 403 con el mismo formato JSON <code>ErrorDetail</code> que el resto de los errores, sin exponer detalles internos.
RNF-S10	CORS	Orígenes, métodos y encabezados permitidos definidos por configuración, solo para <code>/api/**</code> .
RNF-S11	Gestión de secretos	La clave de firma JWT y las credenciales de base de datos y correo se leen de variables de entorno; los valores por omisión son solo para desarrollo local.

4.2 Rendimiento

ID	Requerimiento
RNF-R01	Tiempo de respuesta menor a 5 segundos por transacción con una carga de hasta 5 transacciones por segundo.
RNF-R02	Pool de conexiones HikariCP limitado a 5 conexiones, con tiempo de espera de 20 segundos.
RNF-R03	Los listados de productos y recetas de la API se entregan paginados (20 elementos por omisión, 100 como máximo).
RNF-R04	Asociaciones con carga perezosa; los reportes usan consultas con proyección a DTO y el recetario obtiene el promedio de todas las recetas de la página en una sola consulta.
RNF-R05	Las llamadas a Open Food Facts tienen tiempo límite de 3 segundos de conexión y 5 de lectura; su falla no bloquea el alta de productos.
RNF-R06	Un interceptor registra en bitácora las peticiones que tardan más de 2 segundos.

4.3 Validaciones

ID	Requerimiento
RNF-V01	Toda entrada se valida en el servidor con Bean Validation sobre DTOs, tanto en formularios como en la API.

ID	Requerimiento
RNF-V02	Las reglas que dependen de varios campos o de la base de datos se implementan con validadores de Spring (<code>Validator</code>).
RNF-V03	Los datos de entrada se normalizan antes de validarse: correo en minúsculas y nombres con formato de título.
RNF-V04	Los errores de validación se muestran junto al campo en la web y como lista <code>detalles</code> con código 400 en la API.

4.4 Usabilidad y compatibilidad

ID	Requerimiento
RNF-U01	Interfaz responsive que se adapta a computadora, tableta y teléfono (Bootstrap 5).
RNF-U02	Varios estilos visuales de refrigerador y alacena para que la representación se parezca al mobiliario real.
RNF-U03	Mensajes de confirmación y de error en español después de cada operación (patrón Post/Redirect/Get con mensajes flash).
RNF-U04	Compatibilidad con las versiones vigentes de Chrome, Edge y Firefox.

4.5 Mantenibilidad y buenas prácticas

ID	Requerimiento
RNF-M01	Arquitectura en capas (controlador, servicio, repositorio) con inyección de dependencias, por constructor en todo el código nuevo.
RNF-M02	Separación entre entidades y DTOs; conversión mediante clases Mapper por recurso.
RNF-M03	Transacciones declarativas con <code>@Transactional</code> ; <code>readOnly = true</code> en consultas.
RNF-M04	Manejo centralizado de excepciones: <code>@RestControllerAdvice</code> para la API y página de error para la web.
RNF-M05	API versionada bajo <code>/api/v1</code> , con códigos de estado HTTP y verbos conforme a los principios REST.
RNF-M06	27 pruebas automatizadas: unitarias de servicios con JUnit 5 y Mockito, y de integración de las dos cadenas de seguridad con MockMvc y <code>spring-security-test</code> , sobre una base de datos de pruebas aparte.
RNF-M07	Auditoría de fechas de creación y modificación de recetas con Spring Data JPA Auditing.
RNF-M08	Configuración externalizada en <code>application.properties</code> con valores sensibles por variable de entorno.

5. Arquitectura del sistema

5.1 Vista general

NUBOM es un monolito en capas que corre como una sola aplicación Spring Boot con Tomcat embebido. Se puede entrar por dos lados: los controladores MVC, que regresan vistas de Thymeleaf, y los controladores REST, que regresan JSON. Antes de llegar a cualquiera de ellos la petición pasa por la cadena de filtros de Spring Security que le toca según su ruta. Los dos tipos de controlador llaman a la misma capa de servicios, que es donde están las reglas de negocio y las transacciones, y los servicios llegan a MariaDB por medio de repositorios de Spring Data JPA.

Hacia afuera la aplicación se conecta con dos cosas: la API pública de Open Food Facts, de donde saca datos de productos, y un servidor SMTP con el que manda los avisos de caducidad.

5.2 Tecnologías utilizadas

Capa o propósito	Tecnología	Uso en el proyecto
Lenguaje y plataforma	Java 17, Maven (wrapper)	Records para respuestas de error, API de fechas, expresiones lambda y streams.
Base del proyecto	Spring Boot 3.5.11	Autoconfiguración, servidor embebido y gestión de dependencias.
Núcleo	Spring Framework 6 (IoC)	Beans con estereotipos, inyección por constructor, configuración Java, tareas programadas.
Presentación web	Spring Web MVC, Thymeleaf, Bootstrap 5	Controladores anotados, plantillas con fragmentos, integración de Thymeleaf con Spring Security.
Servicios REST	Spring Web (@RestController), Jackson	API /api/v1, ResponseEntity, manejo global de errores.
Persistencia	Spring Data JPA, Hibernate ORM 6	Repositorios, consultas derivadas, @Query, paginación, proyecciones y auditoría.
Base de datos	MariaDB 10.7, HikariCP	Almacenamiento relacional y pool de conexiones.
Seguridad	Spring Security 6, JWT 0.12	Doble cadena de filtros, BCrypt, JWT, refresh token, CSRF, seguridad a nivel de método.
Validación	Jakarta Bean Validation (Hibernate Validator)	Restricciones en DTOs y validadores propios.
Integración	RestTemplate, Spring Mail	Consumo de Open Food Facts y envío de correos.
Productividad	Lombok, Spring Boot DevTools	Reducción de código repetitivo y recarga en desarrollo.
Pruebas	JUnit 5, Mockito, MockMvc, spring-security-test	Pruebas unitarias y de la capa web con seguridad.
Herramientas	IntelliJ IDEA, Postman, curl	Desarrollo y verificación de la API.

5.3 Diagrama de arquitectura

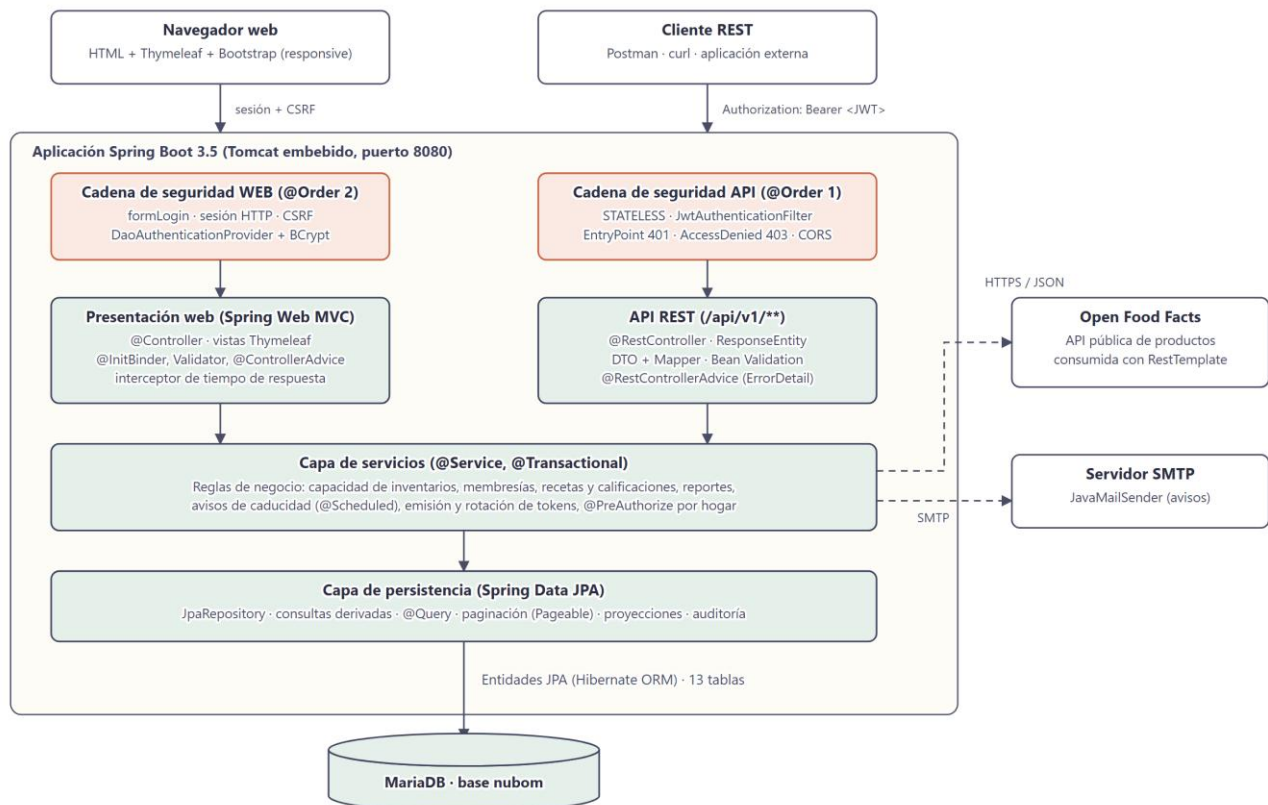


Figura 2. Arquitectura en capas y sistemas externos.

5.4 Estructura de paquetes

dgtic.core	
SpringWebApplication	Clase principal (@EnableScheduling, @EnableJpaAuditing)
config	WebConfig (interceptor), RestClientConfig, SesiónModelAdvice
security	
SecurityConfig	Dos SecurityFilterChain, PasswordEncoder, AuthenticationManager, CORS
CustomUserDetailsService	Usuarios y roles desde la base de datos
UsuarioPrincipal	UserDetails con id, nombre y rol del usuario
UsuarioActual	Acceso al usuario autenticado de la petición
HogarSeguridad	Comprobaciones de pertenencia usadas en @PreAuthorize
jwt	JwtService, JwtAuthenticationFilter, JwtAuthenticationEntryPoint, JwtAccessDeniedHandler, RefreshTokenService, TokenInvalidoException
controller	Controladores MVC: Login, Registro, Inicio, Hogar, ListaCompras, Recetas, Reportes, Notificaciones, Usuario, AdminRecetas
rest	
controller	Controladores REST: Auth, Usuario, Rol, Hogar, InventarioTipo, Inventario, Producto, ListaCompra, Receta, Reporte, Notificacion
dto	DTOs de solicitud y respuesta de la API
mapper	Conversión entidad ⇄ DTO por recurso
service	Servicios propios de la API
exception	Excepciones de negocio, ErrorDetail, GlobalExceptionHandler
service	Reglas de negocio: Usuario, Hogar, Inventario, Producto, ListaCompra, Receta, Notificacion, Reporte, Correo
repository	Repositorios Spring Data JPA
model	
entity	Entidades JPA (sufijo Bd) y enumeraciones
dto	DTOs de formularios, vistas y reportes
validation	Validadores de Spring (Validator)
converter	Editores de propiedades: correo normalizado, formato de título
interceptor	TiempoRespuestaInterceptor
client	OpenFoodFactsClient y sus DTOs
scheduler	CaducidadScheduler, LimpiezaTokensScheduler

src/main/resources	
├ paginas	Plantillas Thymeleaf (plantillas/plantilla.html como base)
├ static/css	nubom.css
├ application.properties	
└ data.sql	Datos iniciales (catálogos, usuarios y datos de prueba)
src/test	Pruebas (perfil "test", base nubom_test)

5.5 Flujo de autenticación (JWT + Refresh Token)

La API maneja dos credenciales distintas. El access token es un JWT firmado que va en cada petición y se valida sin ir a la base de datos, y por eso mismo dura poco. El refresh token es una cadena aleatoria que solo sirve para pedir un par nuevo de tokens. Ese sí se guarda en la base de datos, lo que permite revocarlo. Cada vez que se usa queda invalidado y se entrega otro, que es lo que se conoce como rotación.

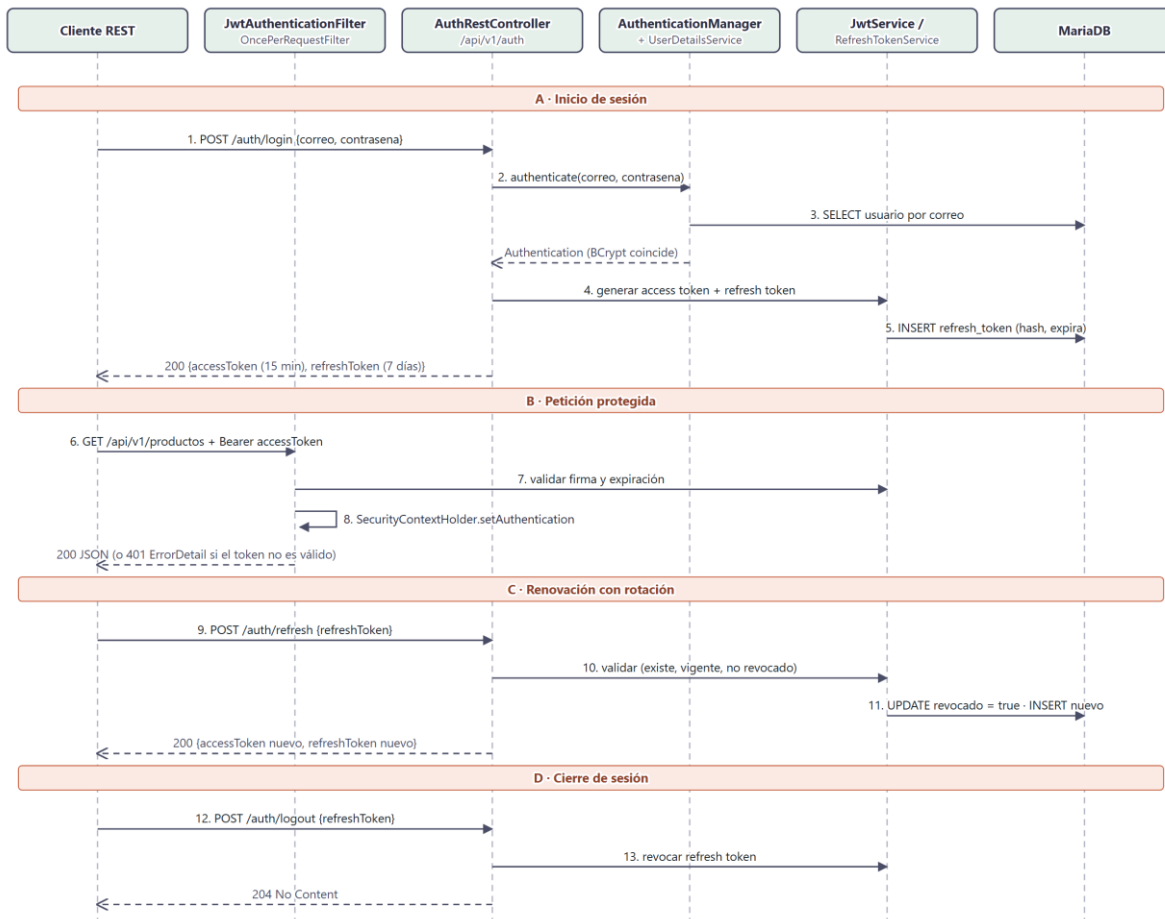


Figura 3. Flujo de autenticación de la API: inicio de sesión, petición protegida, renovación y cierre.

6. Modelo de datos

6.1 Diagrama ER

El modelo tiene 13 tablas en la base de datos nubom de MariaDB. El esquema lo genera Hibernate a partir de las entidades, y el archivo esquema-base-datos.sql lo deja documentado en SQL.

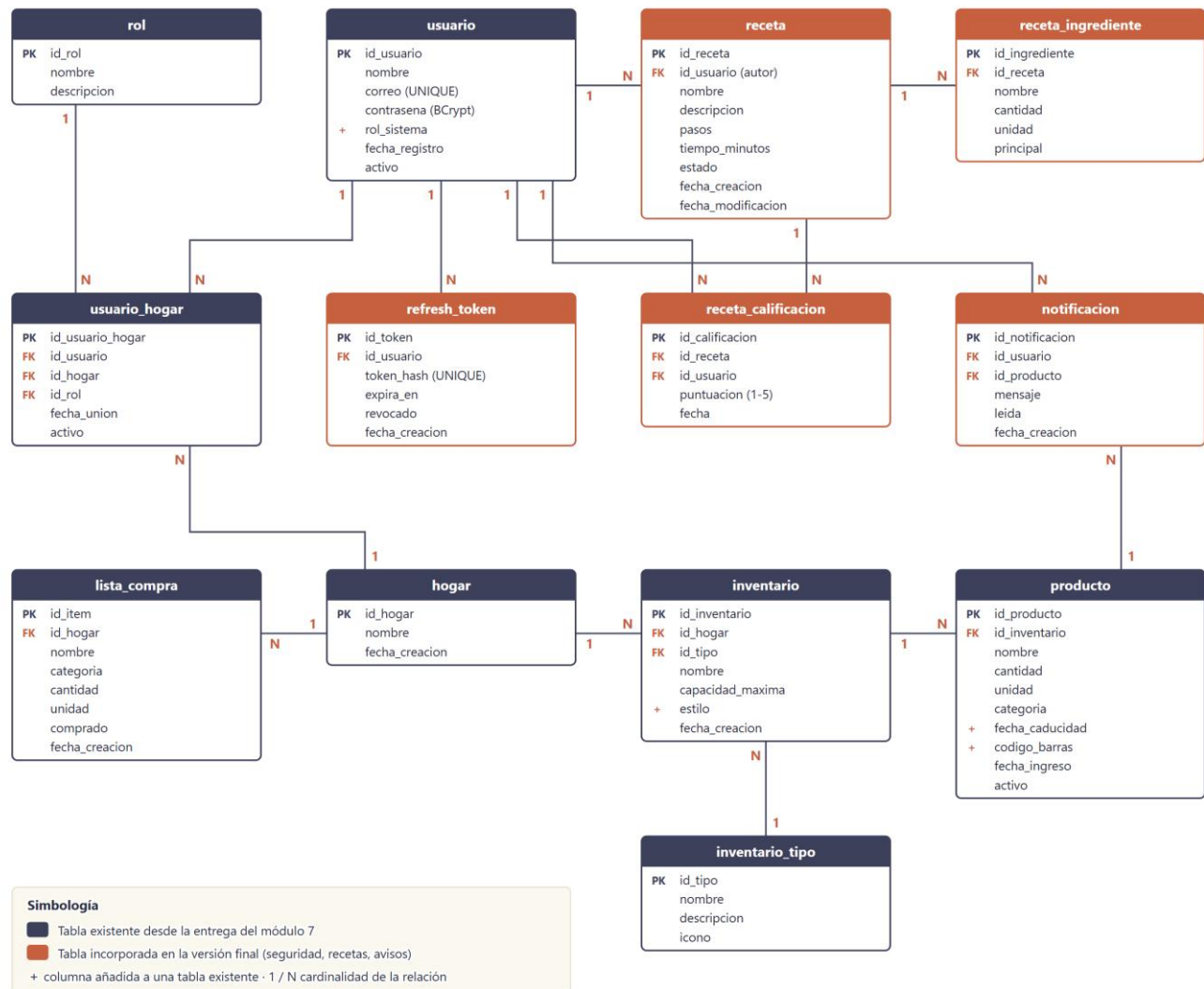


Figura 4. Diagrama entidad-relación.

6.2 Descripción de tablas

Tabla	Propósito	Columnas (PK, FK y restricciones)
usuario	Cuenta de acceso a la aplicación.	id_usuario PK, nombre, correo UNIQUE NOT NULL, contrasena (hash BCrypt, 60 caracteres), rol_sistema (USUARIO ADMIN), fecha_registro, activo
rol	Catálogo de roles dentro de un hogar.	id_rol PK, nombre (PROPIETARIO FAMILIAR), descripcion
hogar	Familia o grupo que comparte inventario.	id_hogar PK, nombre, fecha_creacion

Tabla	Propósito	Columnas (PK, FK y restricciones)
usuario_hogar	Membresía: un usuario y hogar con el rol que tiene en él.	<i>id_usuario_hogar</i> PK, <i>id_usuario</i> FK, <i>id_hogar</i> FK, <i>id_rol</i> FK, <i>fecha_union</i> , <i>activo</i>
inventario_tipo	Catálogo de tipos de almacenamiento.	<i>id_tipo</i> PK, <i>nombre</i> (Refrigerador Alacena), <i>descripcion</i> , <i>icono</i>
inventario	Espacio de almacenamiento de un hogar.	<i>id_inventario</i> PK, <i>id_hogar</i> FK, <i>id_tipo</i> FK, <i>nombre</i> , <i>capacidad_maxima</i> (20, 40 u 80), <i>estilo</i> , <i>fecha_creacion</i>
producto	Alimento guardado en un inventario.	<i>id_producto</i> PK, <i>id_inventario</i> FK, <i>nombre</i> , <i>cantidad</i> , <i>unidad</i> , <i>categoria</i> , <i>fecha_caducidad</i> (opcional), <i>codigo_barras</i> (opcional), <i>fecha_ingreso</i> , <i>activo</i>
lista_compra	Artículo pendiente o ya comprado de un hogar.	<i>id_item</i> PK, <i>id_hogar</i> FK, <i>nombre</i> , <i>categoria</i> , <i>cantidad</i> , <i>unidad</i> , <i>comprado</i> , <i>fecha_creacion</i>
receta	Receta creada por un usuario.	<i>id_receta</i> PK, <i>id_usuario</i> FK (autor), <i>nombre</i> , <i>descripcion</i> , <i>pasos</i> (TEXT), <i>tiempo_minutos</i> , <i>estado</i> (PRIVADA PENDIENTE APROBADA RECHAZADA), <i>fecha_creacion</i> , <i>fecha_modificacion</i>
receta_ingredientes	Ingrediente de una receta.	<i>id_ingredientes</i> PK, <i>id_receta</i> FK, <i>nombre</i> , <i>cantidad</i> , <i>unidad</i> , <i>principal</i> (booleano)
receta_calificacion	Puntuación de un usuario a una receta.	<i>id_calificacion</i> PK, <i>id_receta</i> FK, <i>id_usuario</i> FK, <i>puntuacion</i> (1 a 5), <i>fecha</i> , UNIQUE (<i>id_receta</i> , <i>id_usuario</i>)
notificacion	Aviso de caducidad dirigido a un usuario.	<i>id_notificacion</i> PK, <i>id_usuario</i> FK, <i>id_producto</i> FK, <i>mensaje</i> , <i>leida</i> , <i>fecha_creacion</i>
refresh_token	Refresh tokens emitidos para la API.	<i>id_token</i> PK, <i>id_usuario</i> FK, <i>token_hash</i> UNIQUE (SHA-256), <i>expira_en</i> , <i>revocado</i> , <i>fecha_creacion</i>

Los usuarios y los productos se dan de baja de forma lógica, con la columna *activo*, para no perder el historial. En cambio los hogares, los inventarios, los artículos de la lista de compras y las recetas sí se borran físicamente, junto con lo que dependa de ellos.

7. Diseño del sistema

7.1 Diagramas UML

Secuencia: inicio de sesión en la interfaz web

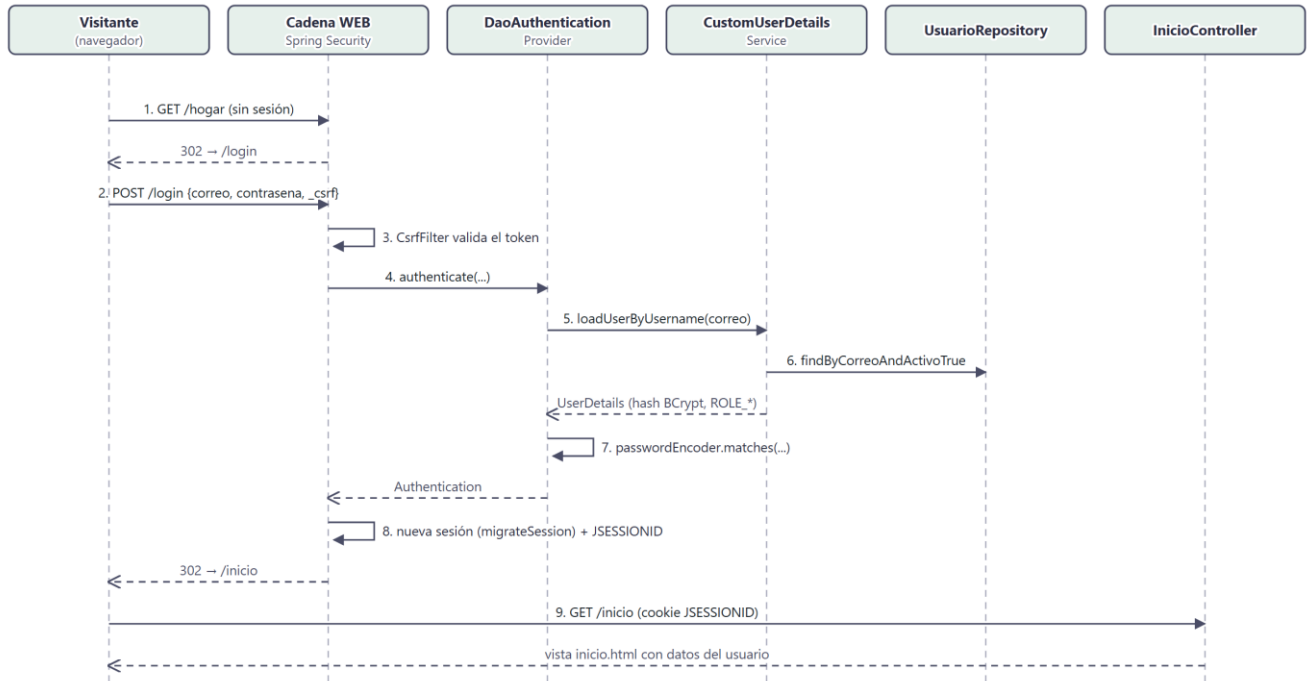


Figura 5. Secuencia de autenticación por formulario.

Secuencia: alta de producto con autocompletado

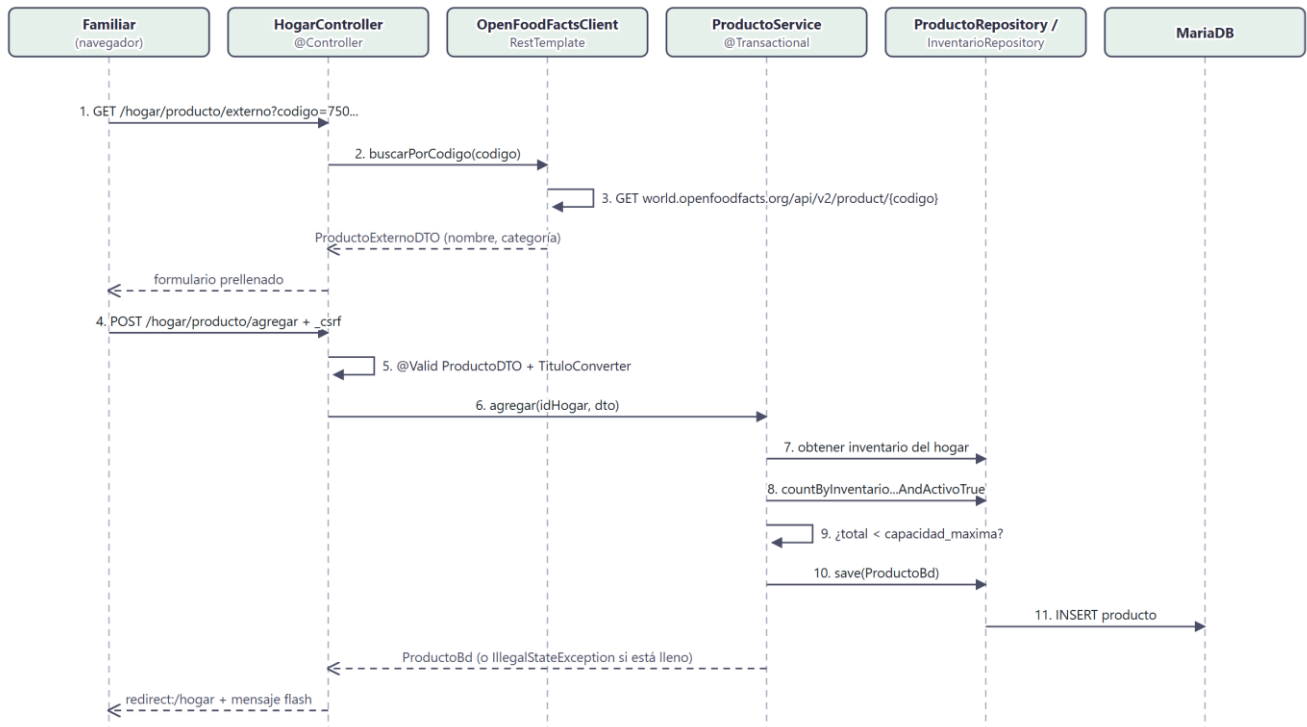


Figura 6. Secuencia del caso de uso CU-05, incluida la consulta a Open Food Facts.

Actividad: generación de avisos de caducidad

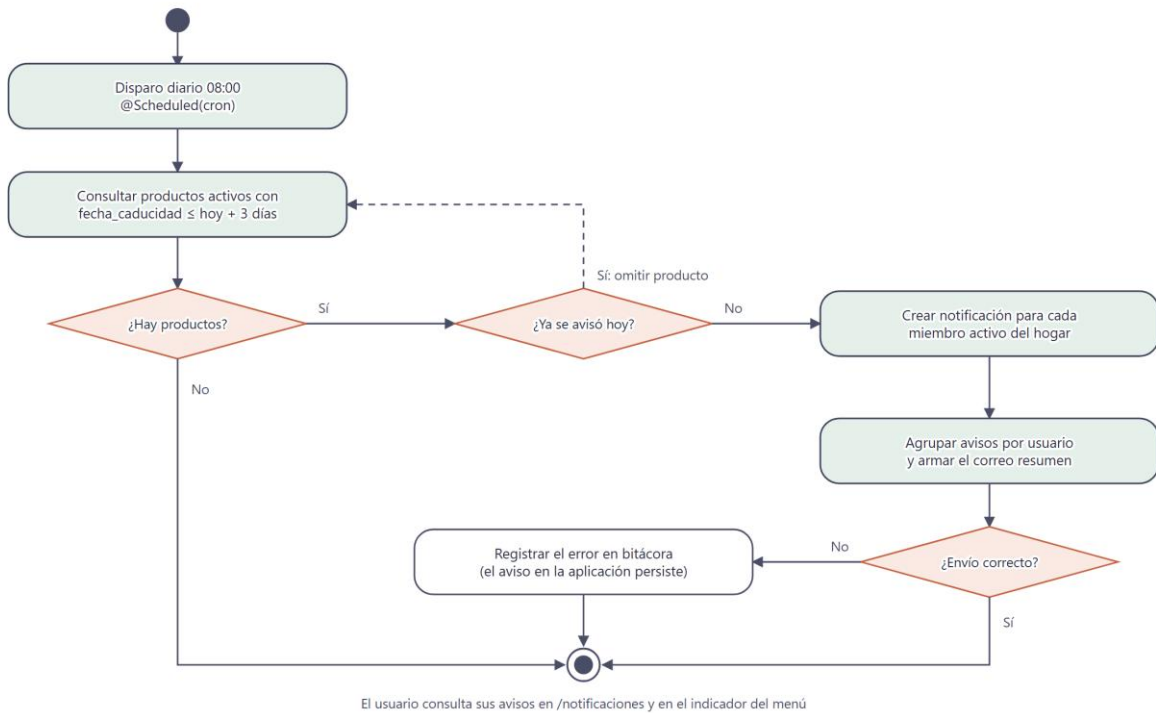


Figura 7. Diagrama de actividad de la tarea programada de caducidad.

Clases: componentes de seguridad

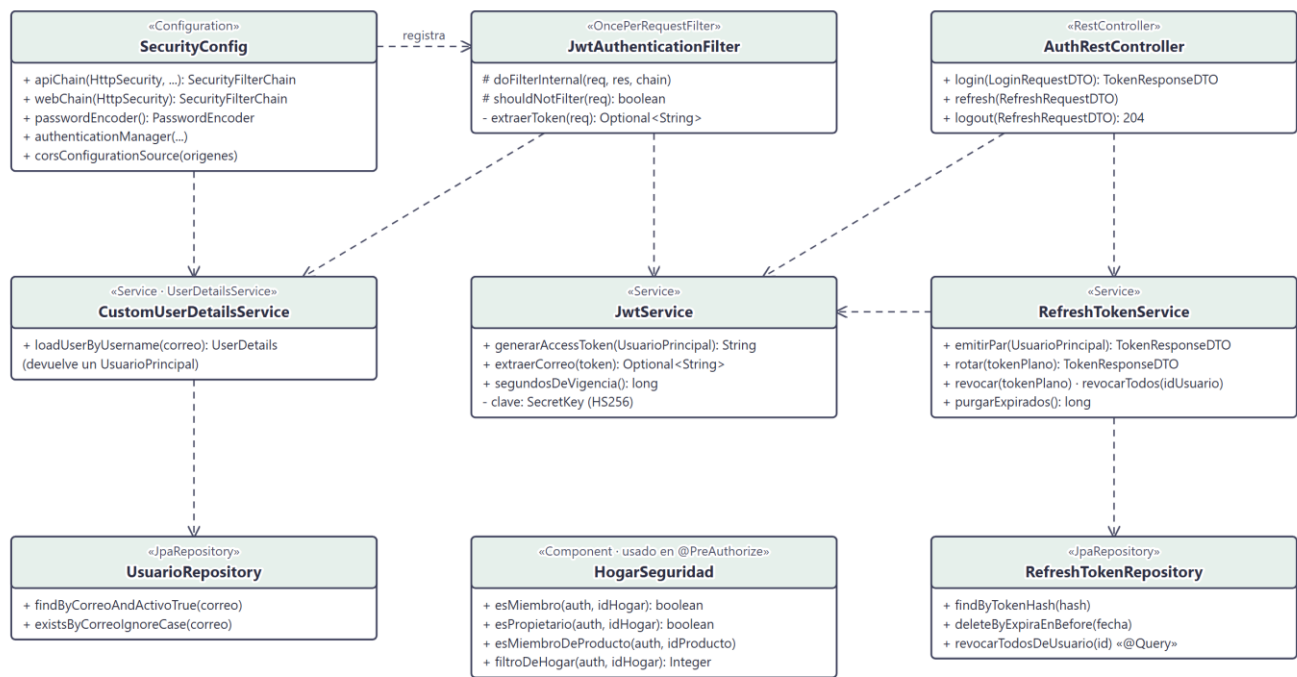


Figura 8. Diagrama de clases de los componentes de seguridad y sus dependencias.

Las clases del dominio corresponden una a una con las tablas de la figura 4. Cada tabla tiene su entidad con el sufijo Bd (por ejemplo ProductoBd) y las asociaciones son las descritas en la sección 3.6.

7.2 DTOs

Ningún controlador recibe ni regresa entidades JPA. Hay dos grupos de DTOs. Unos son los de formulario y vista, que usa Thymeleaf. Los otros son los de la API, que separé en solicitud y respuesta para decidir qué campos se aceptan y cuáles se muestran. La contraseña, por ejemplo, nunca aparece en una respuesta.

Paquete	DTOs	Propósito
model.dto	RegistroUsuarioDTO, PerfilUsuarioDTO, CambiarContrasenaDTO	Formularios de cuenta.
model.dto	HogarDTO, HogarResumenDTO, MiembroHogarDTO	Formulario del hogar y datos para sus vistas (rol del usuario actual, total de miembros).
model.dto	ProductoDTO, ProductoVistaDTO, ListaCompraDTO	Formularios de producto y lista de compras; vista con días para caducar.
model.dto	RecetaDTO, IngredienteDTO, RecetaVistaDTO	Receta con su lista de ingredientes (formulario web y cuerpo de la API); tarjeta con autor y promedio.
model.dto	ProductoCaducidadDTO, ReporteCategoriaDTO, ReporteOcupacionDTO, RecetaRankingDTO	Filas de los cuatro reportes.
rest.dto	LoginRequestDTO, RefreshRequestDTO, TokenResponseDTO	Autenticación de la API.
rest.dto	UsuarioRequestDTO, UsuarioUpdateDTO, UsuarioResponseDTO, RolRequestDTO, RolResponseDTO, MembresiaRequestDTO, MembresiaResponseDTO	Usuarios, roles y membresías.

Paquete	DTOs	Propósito
rest.dto	HogarRequestDTO / ResponseDTO, InventarioTipoRequestDTO / ResponseDTO, InventarioRequestDTO / ResponseDTO	Hogares e inventarios.
rest.dto	ProductoCreateDTO, ProductoRequestDTO, ProductoResponseDTO, ListaCompraRequestDTO / ResponseDTO	Productos y lista de compras.
rest.dto	RecetaResponseDTO, CalificacionRequestDTO, ModeracionRequestDTO, NotificacionResponseDTO, PaginaDTO (record)	Recetas, calificaciones, moderación y avisos; sobre común de las respuestas paginadas.
client	OpenFoodFactsRespuestaDTO, ProductoExternoDTO	Respuesta de la API externa y su versión simplificada.
rest.exception	ErrorDetail (record)	Cuerpo único de error: fecha, estado, error, mensaje, ruta y detalles.

7.3 Validaciones

Dato	Regla	Mecanismo
Nombre de usuario, hogar, producto, receta	Obligatorio; hogar entre 3 y 100 caracteres	@NotBlank, @Size
Correo	Obligatorio, formato válido, único sin distinguir mayúsculas	@Email + RegistroUsuarioValidator
Contraseña	Obligatoria, mínimo 8 caracteres, igual a su confirmación	@Size(min = 8) + Validator
Cambio de contraseña	La actual debe coincidir con el hash guardado; la nueva, con su confirmación	CambiarContraseñaValidator + PasswordEncoder.matches
Cantidad	Obligatoria, mínimo 1	@NotNull, @Min(1)
Unidad y categoría	Obligatorias	@NotBlank; la interfaz web las ofrece en listas cerradas
Fecha de caducidad	Opcional; en el alta no puede ser anterior a hoy	@DateTimeFormat + regla en el servicio (al editar sí se admite una fecha pasada)
Código de barras	Opcional; de 8 a 13 dígitos	@Pattern
Capacidad de inventario	20, 40 u 80 y no menor que los productos activos	Regla en InventarioService
Ingredientes de receta	Al menos uno marcado como principal; cada uno con nombre, cantidad y unidad	@Valid en la lista + RecetaValidator (web) y la misma regla en RecetaService (API)
Puntuación	Entero de 1 a 5	@Min(1), @Max(5)
Parámetros de ruta y consulta	Tipo correcto y presencia de los obligatorios	Manejadores de GlobalExceptionHandler
Entrada de formularios	Correo en minúsculas sin espacios; nombres con formato de título	@InitBinder con CorreoNormalizadoConverter y TituloConverter

7.4 Servicios y repositorios

Servicio	Responsabilidades principales
UsuarioService	Registro con contraseña cifrada, perfil, cambio de contraseña, baja lógica y verificación de que el usuario no sea propietario.
HogarService	Creación del hogar con su membresía de propietario e inventarios iniciales, renombrado, eliminación, integrantes.
InventarioService	Inventarios por hogar, cambio de capacidad y de estilo.

Servicio	Responsabilidades principales
ProductoService	CRUD de productos con verificación de pertenencia y de capacidad; cálculo del estado de caducidad.
ListaCompraService	CRUD de la lista de compras y marcado de comprado.
RecetaService	CRUD de recetas con ingredientes, cambio de estado (compartir, aprobar, rechazar), recetario paginado y calificaciones (alta, cambio, retiro y promedio).
NotificacionService, CorreoService	Generación de avisos, consulta y marcado como leídos; envío del correo resumen, que puede desactivarse por configuración y cuya falla solo se registra en bitácora.
ReporteService	Los cuatro reportes, a partir de consultas de agregación.
JwtService, RefreshTokenService	Emisión y validación de JWT; emisión, rotación, revocación y depuración de refresh tokens.
Servicios REST (<i>rest.service</i>)	Un servicio por recurso de la API: convierten DTOs con su Mapper y traducen los casos de error a excepciones de negocio.

Los servicios de la parte web que ya existían desde entregas anteriores conservan su interfaz y su implementación. Los nuevos (recetas, avisos, reportes y correo) son clases concretas, porque solo tienen una implementación y la interfaz no aportaba nada.

Todos los repositorios extienden `JpaRepository`. En la tabla aparecen las técnicas de Spring Data que usé, con un ejemplo de cada una.

Técnica	Ejemplo
Consulta derivada con propiedades anidadas y orden	<code>findByInventario_Hogar_IdHogarAndActivoTrueOrderByFechaIngresoDesc(idHogar)</code>
Comparación de fechas	<code>findByActivoTrueAndFechaCaducidadLessThanEqual(LocalDate limite)</code>
Conteo y existencia	<code>countByInventario_IdInventarioAndActivoTrue(id), existsByCorreoIgnoreCase(correo)</code>
Operadores de cadena y paginación	<code>Page<RecetaBd> findByEstadoAndNombreContainingIgnoreCase(estado, texto, Pageable p)</code>
@Query con filtros opcionales y paginación	<code>Page<ProductoBd> buscar(idHogar, idInventario, categoria, Pageable p): cada filtro se ignora si llega nulo.</code>
@Query JPQL con proyección a DTO	<code>select new ...MiembroHogarDTO(uh.usuario.nombre, uh.usuario.correo, uh.rol.nombre) from usuario_hogar uh where ...</code>
@Query con agregación	Promedio y número de calificaciones por receta (<code>avg, count, group by</code>); productos por categoría.
Consulta de modificación	<code>@Modifying @Query("update refresh_token t set t.revocado = true where t.usuario.idUsuario = :id")</code>
Limitación de resultados	<code>mejorCalificadas(PageRequest.of(0, 10))</code> para el reporte de recetas mejor calificadas.
Auditoría	<code>@CreatedDate</code> y <code>@LastModifiedDate</code> en <code>RecetaBd</code> con <code>AuditingEntityListener</code> .

8. Seguridad

8.1 Configuración de Spring Security

La clase `SecurityConfig` declara dos cadenas de filtros. Spring Security las revisa en orden y aplica la primera cuyo `securityMatcher` coincide con la ruta. Así el navegador y los clientes de la API reciben cada uno su propio mecanismo y no se estorban:

Aspecto	Cadena API (@Order(1), /api/**)	Cadena WEB (@Order(2), resto de rutas)
Autenticación	JWT en el encabezado <code>Authorization: Bearer</code>	Formulario propio en <code>/login</code>
Estado	Sin sesión (<code>STATELESS</code>)	Sesión HTTP con cookie <code>JSESSIONID</code>
CSRF	Deshabilitado	Habilitado
Sin autenticar	401 con JSON <code>ErrorDetail</code>	Redirección a <code>/login</code>
Sin permiso	403 con JSON <code>ErrorDetail</code>	Página de acceso denegado
Rutas públicas	<code>/api/v1/auth/**</code>	<code>/</code> , <code>/login</code> , <code>/registro</code> , recursos estáticos
Cierre de sesión	Revocación del refresh token	<code>POST /logout</code> : invalida la sesión y borra la cookie

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean @Order(1)
    SecurityFilterChain apiChain(HttpSecurity http, JwtService jwtService,
                                CustomUserDetailsService userDetailsService,
                                JwtAuthenticationEntryPoint entryPoint,
                                JwtAccessDeniedHandler accessDeniedHandler) throws Exception {
        JwtAuthenticationFilter jwtFilter = new JwtAuthenticationFilter(jwtService, userDetailsService);

        return http.securityMatcher("/api/**")
            .csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET,
                    "/api/v1/tipos-inventario/**", "/api/v1/roles/**").authenticated()
                .requestMatchers("/api/v1/tipos-inventario/**", "/api/v1/roles/**",
                    "/api/v1/recetas/moderacion/**").hasRole("ADMIN")
                .anyRequest().authenticated() // el resto se afina con @PreAuthorize
            ).exceptionHandling(e -> e.authenticationEntryPoint(entryPoint)
                .accessDeniedHandler(accessDeniedHandler))
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean @Order(2)
    SecurityFilterChain webChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(a -> a
                .requestMatchers("/", "/login", "/registro", "/acceso-denegado", "/error",
                    "/css/**", "/js/**", "/img/**", "/favicon.ico").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            ).formLogin(f -> f.loginPage("/login")
                .usernameParameter("correo").passwordParameter("contrasena")
                .defaultSuccessUrl("/inicio", true).failureUrl("/login?error"))
            .logout(l -> l.logoutUrl("/logout").logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true).deleteCookies("JSESSIONID"))
            .sessionManagement(s -> s.sessionFixation(f -> f.migrateSession()).maximumSessions(1))
            .exceptionHandling(e -> e.accessDeniedPage("/acceso-denegado"))
            .build();
    }
}
```

```
@Bean
PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

Usuarios y roles desde la base de datos

CustomUserDetailsService implementa UserDetailsService. Busca por correo al usuario activo y arma un UsuarioPrincipal, que es un UserDetails al que le agregué el id y el nombre, con su hash BCrypt y la autoridad ROLE_USUARIO o ROLE_ADMIN según la columna rol_sistema. El rol dentro del hogar (propietario o familiar) no lo manejo como autoridad global porque depende de cada hogar, así que se revisa en cada operación con el bean HogarSeguridad:

```
// HogarServiceImpl: operación exclusiva del propietario
@PreAuthorize("@hogarSeguridad.esPropietario(authentication, #idHogar)")
public void renombrarHogar(Integer idHogar, Integer idUsuarioSolicitante, HogarDTO dto) { ... }

// ProductoRestController: el producto debe ser de un inventario del hogar del usuario
@GetMapping("/{id}")
@PreAuthorize("@hogarSeguridad.esMiembroDeProducto(authentication, #id)")
public ResponseEntity<ProductoResponseDTO> obtener(@PathVariable("id") Integer id) { ... }

// UsuarioRestController: el administrador o el propio usuario
@PreAuthorize("hasRole('ADMIN') or @hogarSeguridad.esUsuario(authentication, #id)")
public ResponseEntity<UsuarioResponseDTO> obtener(@PathVariable("id") Integer id) { ... }
```

Los listados generales de la API (/productos, /inventarios, /lista-compra y /hogares) nunca regresan datos de otros hogares. Si quien consulta no es administrador, el resultado se filtra por su hogar aunque no mande el parámetro idHogar, y si pide el de otro hogar recibe un 403.

Integración con Thymeleaf

Las plantillas usan el dialecto thymeleaf-extras-springsecurity6 para mostrar el nombre de quien inició sesión y para esconder las opciones que no le corresponden. Un ejemplo es sec:authorize="hasRole('ADMIN')" en el enlace a la bandeja de moderación. Esconder un enlace es solo una ayuda visual, porque la restricción de verdad se aplica en el servidor.

8.2 Filtros personalizados

Componente	Tipo	Función
JwtAuthenticationFilter	OncePerRequestFilter	Se ejecuta antes de UsernamePasswordAuthenticationFilter en la cadena de la API. Se instancia en SecurityConfig (no es un bean) para que Spring Boot no lo registre además en toda la aplicación. Extrae el token del encabezado, valida firma y vigencia con JwtService, carga al usuario y coloca la autenticación en el SecurityContext. Si no hay token deja pasar la petición sin autenticar, y la regla de autorización decide. No se aplica a /api/v1/auth/** (shouldNotFilter).
JwtAuthenticationEntryPoint	AuthenticationEntryPoint	Responde 401 con ErrorDetail cuando falta el token, está vencido o es inválido.
JwtAccessDeniedHandler	AccessDeniedHandler	Responde 403 con ErrorDetail cuando el usuario autenticado no tiene permiso.
TiempoRespuestaInterceptor	HandlerInterceptor (Spring MVC)	No es un filtro de seguridad: mide la duración de cada petición y registra las que superan 2 segundos.

8.3 JWT y Refresh Token

Característica	Access token	Refresh token
----------------	--------------	---------------

Característica	Access token	Refresh token
Formato	JWT firmado con HMAC-SHA256 (HS256)	Cadena aleatoria de 256 bits (opaca)
Contenido	<code>sub</code> (correo), <code>uid</code> , <code>rol</code> , <code>iat</code> , <code>exp</code>	Sin información; es solo un identificador
Vigencia	15 minutos	7 días
Almacenamiento en servidor	Ninguno	Hash SHA-256 en la tabla <code>refresh_token</code>
Uso	En cada petición a la API	Solo en <code>/auth/refresh</code> y <code>/auth/logout</code>
Revocación	No aplica; expira pronto	Inmediata: bandera <code>revocado</code>

- Rotación: cada renovación invalida el refresh token usado y entrega uno nuevo. Si llega un token que ya estaba revocado se toma como posible robo y se revocan todos los refresh tokens de ese usuario.
- Al cambiar la contraseña o dar de baja la cuenta también se revocan todos los refresh tokens del usuario.
- Limpieza: `LimpiezaTokensScheduler` borra cada noche los tokens que ya vencieron.
- Clave de firma: se lee de la variable de entorno `NUBOM_JWT_SECRET` (al menos 256 bits, en Base64). Si no está definida se usa una clave que viene en la configuración y que solo sirve para desarrollo.

```
POST /api/v1/auth/login
{ "correo": "armando@correo.com", "contrasena": "*****" }

200 OK
{ "tokenType": "Bearer",
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresIn": 900,
  "refreshToken": "YnWXXeUjVJcUnITeUBsLcwr347eYZbSTcTS_0..." }
```

8.4 CSRF

En la interfaz web la protección CSRF está activa. El navegador manda la cookie de sesión por su cuenta y un sitio malicioso podría aprovecharse de eso. Thymeleaf agrega el campo oculto `_csrf` a todos los formularios que usan `th:action`, y Spring Security rechaza con 403 cualquier petición que modifique datos y no lo traiga (figura E21). Por lo mismo, todo lo que modifica datos se hace con POST, incluido el cierre de sesión.

En la API está desactivada. Ahí no hay cookies ni sesión, y el token se tiene que poner a mano en el encabezado `Authorization`, cosa que un sitio de terceros no puede hacer a nombre del usuario.

8.5 Matriz de autorización

Operación	Visitante	Familiar	Propietario	Administrador
Registrarse, iniciar sesión	Sí	—	—	—
Perfil propio, cambio de contraseña, baja	No	Sí	Sí (baja bloqueada)	Sí
Consultar inventario, productos y lista de compras de su hogar	No	Sí	Sí	Solo listados generales por la API
Alta, cambio y baja de productos y lista de compras de su hogar	No	Sí	Sí	No
Renombrar o eliminar el hogar, gestionar integrantes	No	No	Sí	No
Cambiar capacidad y estilo de inventarios	No	No	Sí	No
CRUD de recetas propias, compartir, calificar recetas ajenas	No	Sí	Sí	Sí
Aprobar o rechazar recetas compartidas	No	No	No	Sí
Reportes de su hogar y avisos propios	No	Sí	Sí	No

Operación	Visitante	Familiar	Propietario	Administrador
Catálogos (lectura)	No	Sí	Sí	Sí
Catálogos (escritura) y administración de usuarios	No	No	No	Sí

9. API REST

9.1 Documentación de endpoints

La ruta base es `http://localhost:8080/api/v1` y todo viaja como JSON en UTF-8. Con excepción de los de autenticación, los endpoints piden el encabezado `Authorization: Bearer <accessToken>`. En la columna «Acceso», Miembro y Propietario se refieren al hogar del recurso que se está pidiendo.

Autenticación

Método	Ruta	Descripción	Acceso
POST	/auth/login	Valida credenciales y entrega access token y refresh token.	Público
POST	/auth/refresh	Entrega un nuevo par de tokens y revoca el refresh token usado.	Público (requiere refresh token)
POST	/auth/logout	Revoca el refresh token. Responde 204.	Público (requiere refresh token)

Usuarios, roles y membresías

Método	Ruta	Descripción	Acceso
GET	/usuarios	Lista usuarios.	Administrador
GET	/usuarios/{id}	Consulta un usuario.	Administrador o el propio usuario
POST	/usuarios	Crea un usuario; la contraseña se guarda cifrada.	Administrador
PUT	/usuarios/{id}	Actualiza nombre y correo.	Administrador o el propio usuario
DELETE	/usuarios/{id}	Elimina al usuario; 409 si tiene datos relacionados (membresías, recetas, avisos).	Administrador
GET	/usuarios/{idUserario}/hogares	Hogares a los que pertenece el usuario.	Administrador o el propio usuario
POST	/usuarios/{idUserario}/hogares/{idHogar}	Agrega al usuario al hogar con un rol.	Propietario
DELETE	/usuarios/{idUserario}/hogares/{idHogar}	Retira al usuario del hogar.	Propietario
GET	/roles, /roles/{id}	Consulta el catálogo de roles de hogar.	Autenticado
POST	/roles	Crea un rol.	Administrador
PUT	/roles/{id}	Actualiza un rol.	Administrador
DELETE	/roles/{id}	Elimina un rol; 409 si está en uso.	Administrador

Hogares e inventarios

Método	Ruta	Descripción	Acceso
GET	/hogares	Lista los hogares del usuario (todos si es administrador).	Autenticado
GET	/hogares/{id}	Consulta un hogar.	Miembro
POST	/hogares	Crea un hogar con sus dos inventarios y deja al usuario como propietario. 201 con <code>Location</code> ; 409 si ya pertenece a un hogar.	Autenticado

Método	Ruta	Descripción	Acceso
PUT	/hogares/{id}	Renombra el hogar.	Propietario
DELETE	/hogares/{id}	Elimina el hogar y sus dependientes.	Propietario
GET	/hogares/{id}/inventarios	Inventarios del hogar.	Miembro
POST	/hogares/{id}/inventarios	Crea un inventario adicional.	Propietario
GET	/hogares/{id}/lista-compra	Lista de compras del hogar.	Miembro
POST	/hogares/{id}/lista-compra	Agrega un artículo a la lista.	Miembro
GET	/hogares/{id}/usuarios	Integrantes del hogar con su rol.	Miembro
GET	/inventarios, /inventarios/{id}	Consulta inventarios del hogar del usuario; filtro opcional <code>idHogar</code> .	Miembro
PUT	/inventarios/{id}	Actualiza nombre, capacidad y estilo.	Propietario
DELETE	/inventarios/{id}	Elimina el inventario y sus productos.	Propietario
GET	/inventarios/{id}/productos	Productos activos del inventario.	Miembro
POST	/inventarios/{id}/productos	Agrega un producto; 409 si el inventario está lleno.	Miembro
GET	/tipos-inventario, /tipos-inventario/{id}	Consulta el catálogo de tipos.	Autenticado
POST	/tipos-inventario	Crea un tipo.	Administrador
PUT	/tipos-inventario/{id}	Actualiza un tipo.	Administrador
DELETE	/tipos-inventario/{id}	Elimina un tipo; 409 si está en uso.	Administrador

Productos y lista de compras

Método	Ruta	Descripción	Acceso
GET	/productos	Lista paginada. Parámetros: <code>idHogar</code> , <code>idInventario</code> , <code>categoria</code> , <code>page</code> , <code>size</code> , <code>sort</code> .	Miembro
GET	/productos/{id}	Consulta un producto.	Miembro
POST	/productos	Crea un producto indicando <code>idInventario</code> .	Miembro
PUT	/productos/{id}	Actualiza un producto.	Miembro
DELETE	/productos/{id}	Baja lógica del producto. Responde 204.	Miembro
GET	/productos/externo/{codigoBarras}	Datos sugeridos del producto desde Open Food Facts.	Autenticado
GET	/lista-compra, /lista-compra/{id}	Consulta artículos; filtro opcional <code>idHogar</code> .	Miembro
PUT	/lista-compra/{id}	Actualiza un artículo, incluido <code>comprado</code> .	Miembro
DELETE	/lista-compra/{id}	Elimina un artículo.	Miembro

Recetas, avisos y reportes

Método	Ruta	Descripción	Acceso
GET	/recetas	Recetario de la comunidad (aprobadas), paginado; parámetro <code>nombre</code> .	Autenticado
GET	/recetas/mias	Recetas del usuario en cualquier estado.	Autenticado
GET	/recetas/{id}	Detalle con ingredientes y promedio.	Autor, o cualquiera si está aprobada
POST	/recetas	Crea una receta con sus ingredientes.	Autenticado
PUT	/recetas/{id}	Actualiza la receta y reemplaza sus ingredientes.	Autor

Método	Ruta	Descripción	Acceso
DELETE	/recetas/{id}	Elimina la receta.	Autor
POST	/recetas/{id}/compartir	Solicita publicar la receta (pasa a PENDIENTE).	Autor
PUT	/recetas/{id}/calificacion	Registra o cambia la calificación del usuario.	Autenticado, no autor
DELETE	/recetas/{id}/calificacion	Retira la calificación del usuario.	Autenticado
GET	/recetas/moderacion	Recetas pendientes de revisión.	Administrador
PUT	/recetas/moderacion/{id}	Aprueba o rechaza una receta.	Administrador
GET	/notificaciones	Avisos del usuario; parámetro soloNoLeidas.	Autenticado
PUT	/notificaciones/{id}/leida	Marca un aviso como leído.	Destinatario
GET	/reportes/por-caducar	Productos del hogar que vencen en los próximos días.	Miembro
GET	/reportes/por-categoria	Número de productos por categoría.	Miembro
GET	/reportes/ocupacion	Productos, capacidad y porcentaje por inventario.	Miembro
GET	/reportes/recetas-top	Las 10 recetas mejor calificadas.	Autenticado

Respuestas paginadas

Los listados paginados (/productos y /recetas) aceptan page (empieza en 0), size (20 si no se indica y 100 como máximo) y sort (propiedad,asc|desc). Todos responden con la misma estructura:

```
{ "contenido": [ ... ], "pagina": 0, "tamano": 20, "totalElementos": 15, "totalPaginas": 1 }
```

Códigos de estado y formato de error

Código	Situación
200 OK, 201 Created, 204 No Content	Consulta o actualización correcta, recurso creado (con encabezado Location), eliminación correcta.
400 Bad Request	Datos o parámetros inválidos, JSON mal formado.
401 Unauthorized	Falta el token, está vencido o las credenciales son incorrectas.
403 Forbidden	El usuario no tiene el rol requerido o no pertenece al hogar del recurso.
404 Not Found	El recurso no existe.
409 Conflict	La operación viola una regla de integridad: correo duplicado, inventario lleno, catálogo en uso.
500, 503	Error inesperado, servicio externo no disponible.

```
{ "timestamp": "2026-10-08T10:15:30",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Los datos enviados no son válidos",
  "ruta": "/api/v1/productos",
  "detalles": [{"cantidad": "La cantidad debe ser al menos 1"}] }
```

La documentación operación por operación de los recursos que entregué en el módulo 7, con ejemplos de solicitud y respuesta, sigue en docs/documentacion_nubom_api.pdf y docs/guia_pruebas_api_nubom.pdf.

9.2 Consumo de API externa

Para que capturar productos sea más rápido, NUBOM consulta Open Food Facts, una base de datos abierta de productos alimenticios que no pide llave de acceso. Cuando el usuario escribe un código de barras, el sistema le propone el nombre y la categoría, y él puede aceptarlos o corregirlos antes de guardar.

Elemento	Detalle
Servicio	GET https://world.openfoodfacts.org/api/v2/product/{codigo}.json?fields=product_name,brands,categories_tags,quantity
Cliente	<code>OpenFoodFactsClient</code> (<code>@Component</code>) con un bean <code>RestTemplate</code> creado mediante <code>RestTemplateBuilder</code> : URL base, tiempos límite y encabezado <code>User-Agent</code> que identifica a la aplicación.
Conversión	La respuesta se recibe en <code>OpenFoodFactsRespuestaDTO</code> y se reduce a <code>ProductoExternoDTO</code> (nombre, marca, categoría sugerida, presentación). Las etiquetas de categoría del servicio se traducen a las categorías de NUBOM con una tabla de equivalencias.
Exposición	GET <code>/api/v1/productos/externo/{codigoBarras}</code> en la API y botón de búsqueda junto al campo «Código de barras» del formulario web (figura E6).
Manejo de fallas	Producto inexistente: 404. Tiempo agotado o error del servicio: 503 con mensaje claro. En la interfaz web el formulario sigue disponible para captura manual.

```
@Component
public class OpenFoodFactsClient {

    private static final String RUTA =
        "/api/v2/product/{codigo}.json?fields=product_name,brands,categories_tags,quantity";

    private final RestTemplate restTemplate; // bean "openFoodFactsRestTemplate"

    public Optional<ProductoExternoDTO> buscarPorCodigo(String codigo) {
        try {
            OpenFoodFactsRespuestaDTO respuesta =
                restTemplate.getForObject(RUTA, OpenFoodFactsRespuestaDTO.class, codigo);
            if (respuesta == null || respuesta.getStatus() != 1 || respuesta.getProduct() == null) {
                return Optional.empty();
            }
            OpenFoodFactsRespuestaDTO.Producto p = respuesta.getProduct();
            return Optional.of(new ProductoExternoDTO(codigo, p.getProductName().trim(), p.getBrands(),
                categoriaDe(p.getCategoriesTags()), p.getQuantity()));
        } catch (HttpClientErrorException.NotFound ex) {
            return Optional.empty();
        } catch (RestClientException ex) {
            throw new ServicioExternoNoDisponibleException(
                "El servicio de consulta de productos no está disponible; captura los datos manualmente", ex);
        }
    }
}
```

La figura E28 muestra una consulta real. Para el código 3017620422003 el servicio externo regresa «Nutella» y NUBOM sugiere la categoría «Abarrotes».

10. Configuración

Toda la configuración está en `src/main/resources/application.properties`, que se reproduce completo a continuación. Los valores sensibles se toman de variables de entorno, y lo que aparece después de los dos puntos es el valor para desarrollo local. Aquí omití la contraseña de la base de datos y la clave JWT de desarrollo.

```
spring.application.name=nubom

server.port=8080

# Thymeleaf
spring.thymeleaf.prefix=classpath:/paginas/
spring.thymeleaf.suffix=.html
spring.thymeleaf.cache=false

# -----
# Base de datos (MariaDB) y pool de conexiones.
# Los valores sensibles se leen de variables de entorno; lo que aparece
# despues de los dos puntos es el valor para desarrollo local.
# -----
spring.datasource.url=jdbc:mariadb://localhost:3306/nubom?useSSL=false&serverTimezone=UTC&createDatabaseIfNotExist=true
&useUnicode=true&characterEncoding=UTF-8
spring.datasource.username=${NUBOM_DB_USER:root}
spring.datasource.password=${NUBOM_DB_PASSWORD:...}
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.datasource.hikari.connection-timeout=20000
spring.datasource.hikari.maximum-pool-size=5

# JPA / Hibernate. En desarrollo el esquema se recrea en cada arranque y se
# cargan los datos de data.sql. Fuera de desarrollo: ddl-auto=validate y
# spring.sql.init.mode=never.
spring.jpa.hibernate.ddl-auto=create
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
spring.sql.init.encoding=UTF-8
spring.jpa.show-sql=false
spring.jpa.open-in-view=true

# Paginacion de la API
spring.data.web.pageable.default-page-size=20
spring.data.web.pageable.max-page-size=100

# -----
# Sesion de la aplicacion web
# -----
server.servlet.session.timeout=30m
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=lax

# -----
# JWT y refresh token de la API.
# La clave por omision es SOLO para desarrollo: en cualquier otro ambiente
# debe definirse NUBOM_JWT_SECRET (minimo 32 bytes aleatorios en Base64).
# -----
nubom.jwt.secret=${NUBOM_JWT_SECRET:...}
nubom.jwt.access-expiration=15m
nubom.jwt.refresh-expiration=7d
nubom.jwt.limpieza-cron=0 30 3 * * *

# CORS de la API (origenes separados por coma)
nubom.cors.allowed-origins=http://localhost:5173

# -----
# Avisos de caducidad
# -----
nubom.caducidad.dias-aviso=3
nubom.caducidad.cron=0 0 8 * * *
nubom.caducidad.ejecutar-al-iniciar=true
```

```
# -----
# Correo. Desactivado por omision: para enviar correos reales define
# NUBOM_MAIL_ENABLED=true junto con el usuario y la contrasena SMTP.
# -----
nubom.mail.enabled=${NUBOM_MAIL_ENABLED:false}
nubom.mail.from=${NUBOM_MAIL_USER:nubom@localhost}
spring.mail.host=${NUBOM_MAIL_HOST:smtp.gmail.com}
spring.mail.port=587
spring.mail.username=${NUBOM_MAIL_USER:}
spring.mail.password=${NUBOM_MAIL_PASSWORD:}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# -----
# Open Food Facts (API externa de productos)
# -----
nubom.openfoodfacts.base-url=https://world.openfoodfacts.org
nubom.openfoodfacts.connect-timeout=3s
nubom.openfoodfacts.read-timeout=5s

# Bitacoras
logging.level.dgtic.core=INFO
logging.level.org.springframework.security=WARN
```

Las pruebas corren con el perfil test (src/test/resources/application-test.properties), que apunta a la base nubom_test para no tocar los datos de desarrollo.

Dependencias Maven

Dependencia	Propósito
spring-boot-starter-web	Spring Web MVC, Tomcat embebido y Jackson.
spring-boot-starter-thymeleaf, thymeleaf-extras-springsecurity6	Motor de plantillas y su integración con Spring Security.
spring-boot-starter-data-jpa, mariadb-java-client	Persistencia y controlador JDBC.
spring-boot-starter-security	Autenticación y autorización.
jjwt-api, jjwt-impl, jjwt-jackson (0.12.x)	Creación y validación de JWT.
spring-boot-starter-validation	Bean Validation.
spring-boot-starter-mail	Envío de correos.
lombok, spring-boot-devtools	Apoyo al desarrollo.
spring-boot-starter-test, spring-security-test	JUnit 5, Mockito, MockMvc y utilidades de prueba de seguridad.

11. Evidencia de funcionamiento

Las capturas las tomé el 8 de octubre de 2026 con la aplicación corriendo en localhost:8080 y con los datos iniciales de data.sql. Las de la interfaz web son pantallas del navegador. Las de seguridad y de la API muestran la salida real de cur1 y del cliente de MariaDB.

11.1 Flujo de usuario en la interfaz web

1. La persona se registra; los datos se validan en el servidor.
2. Inicia sesión y llega a la pantalla de inicio con el resumen de su hogar.
3. Consulta su refrigerador y su alacena, y agrega productos (con autocompletado por código de barras).
4. Administra la lista de compras y sus recetas; califica las de la comunidad.
5. Consulta reportes y avisos de caducidad.
6. Como propietaria, administra el hogar e incorpora a un familiar, que ve el mismo inventario.

Figura E1. Registro con errores de validación: correo ya registrado, contraseña corta y confirmación distinta

Figura E2. Inicio de sesión con credenciales incorrectas

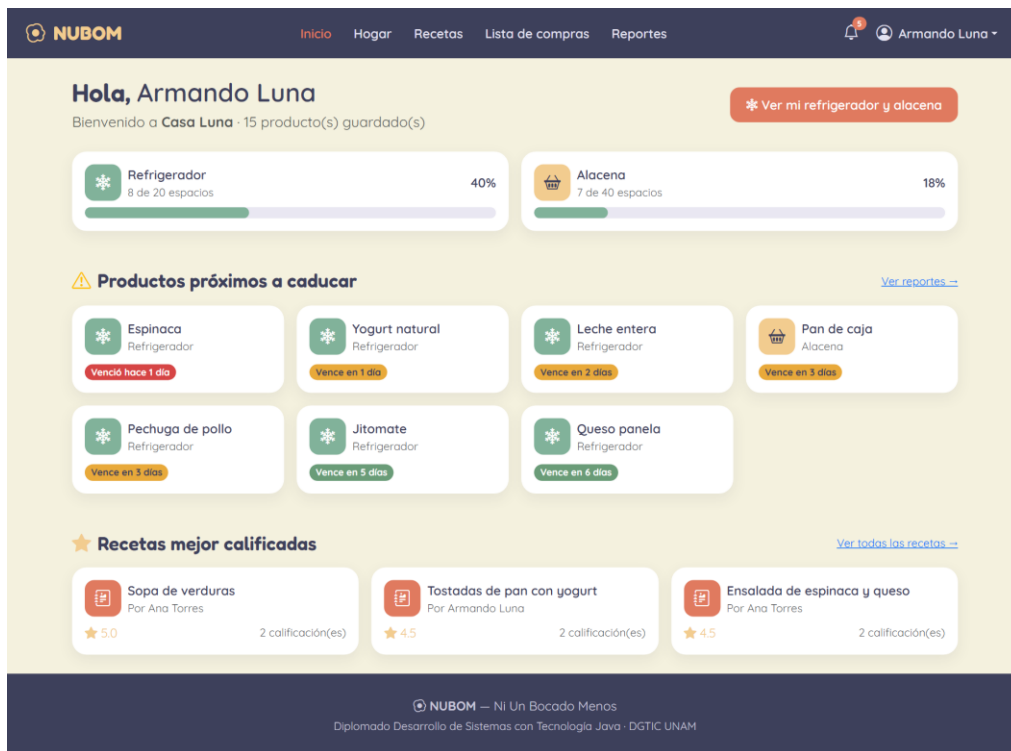


Figura E3. Pantalla de inicio: nivel de ocupación, productos próximos a caducar y recetas mejor calificadas

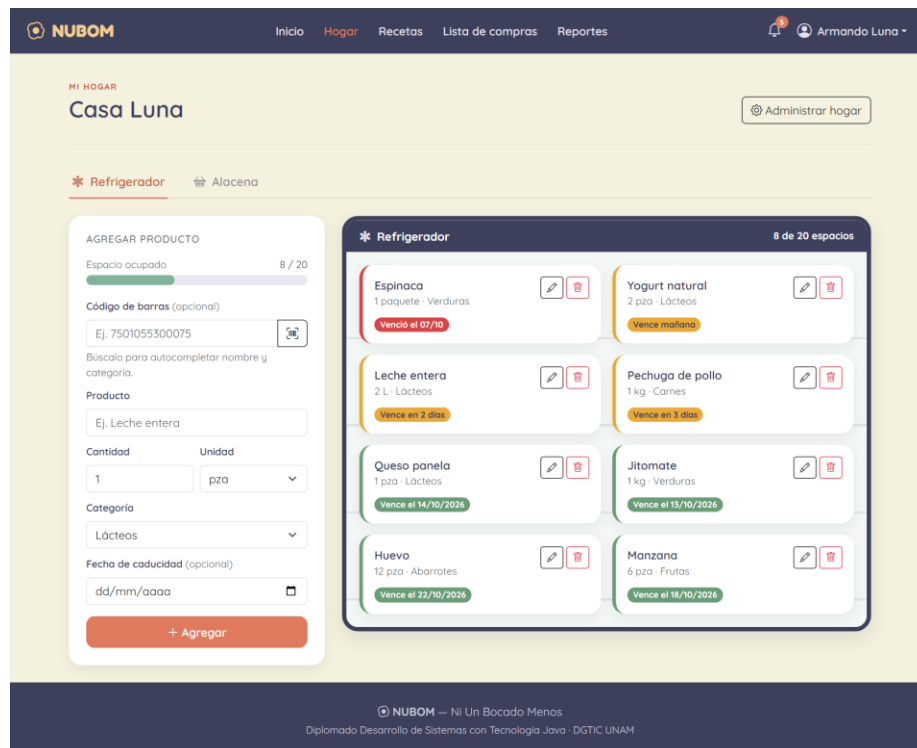


Figura E4. Vista gráfica del refrigerador con el semáforo de caducidad de cada producto

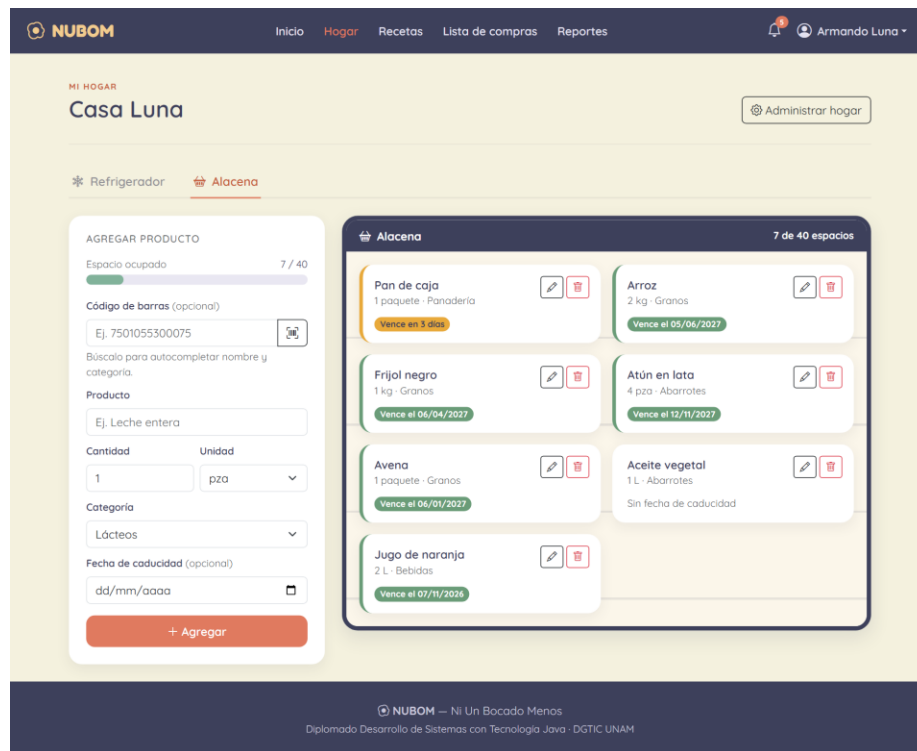


Figura E5. Vista gráfica de la alacena

Código de barras (opcional)

3017620422003



Datos sugeridos por Open Food Facts.
Revísalos antes de guardar.

Producto

Nutella

Cantidad

1

Unidad

pza

Categoría

Abarrotes

Fecha de caducidad (opcional)

dd/mm/aaaa



+ Agregar

Figura E6. Alta de producto: nombre y categoría sugeridos por Open Food Facts a partir del código de barras

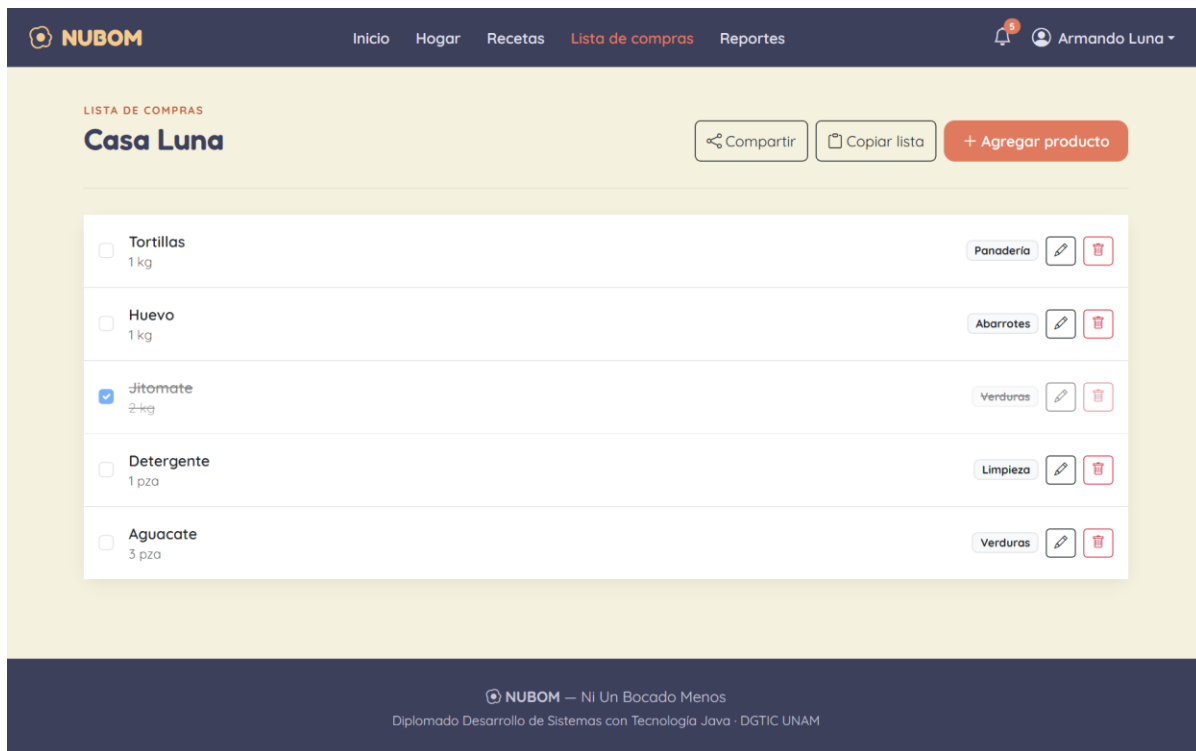


Figura E7. Lista de compras del hogar

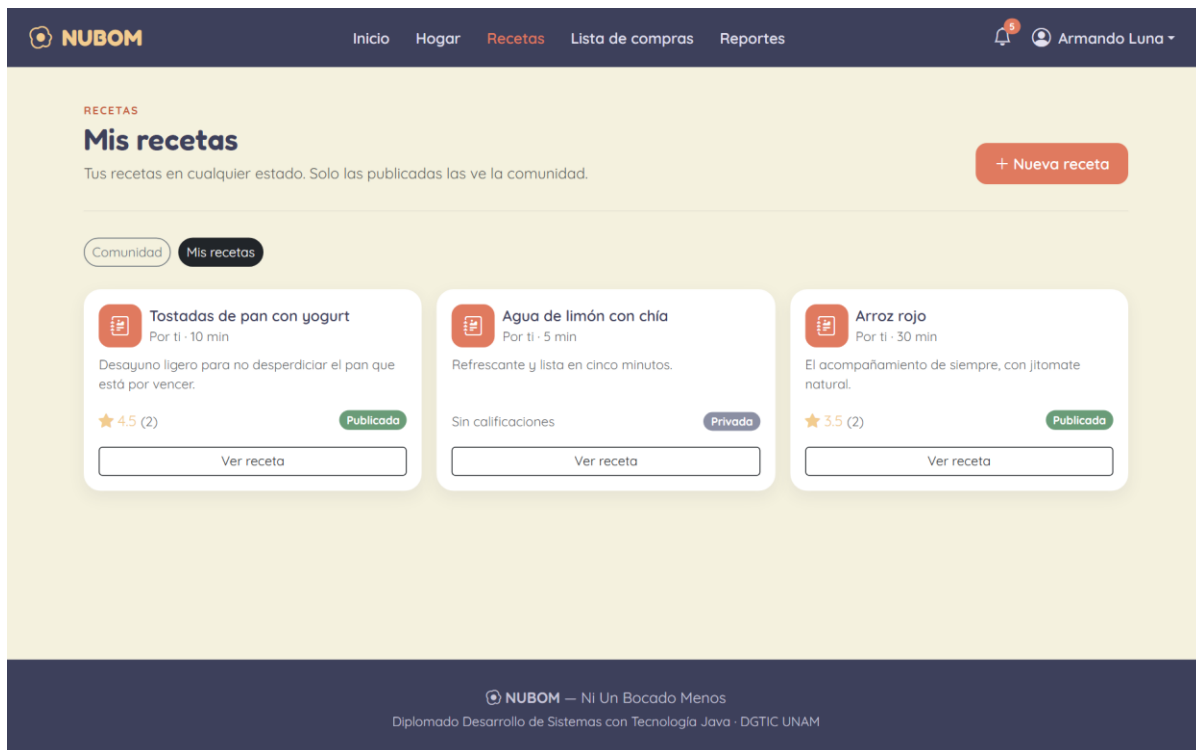


Figura E8. Recetas propias con su estado: publicada, privada o en revisión

Inicio
Hogar
Recetas
Lista de compras
Reportes

Armando Luna

RECETAS

Editar receta

Marca como **principal** los ingredientes sin los que la receta no se puede preparar.

Nombre

Tostadas de pan con yogurt

Descripción

Desayuno ligero para no desperdiciar el pan que está por vencer.

Tiempo (minutos)

10

Pasos de preparación

Tuesta las rebanadas de pan.
Unta una capa de yogurt natural.
Agrega rodajas de manzana y un poco de avena.
Sirve de inmediato.

Un paso por línea; se numeran solos.

Ingredientes

+ Agregar

Pan de caja	4.0	pza	☒ Principal	
Yogurt natural	1.0	pza	☒ Principal	
Manzana	1.0	pza	☐ Principal	
Avena	2.0	cda	☐ Principal	

Se necesita al menos un ingrediente principal.

✓ Guardar receta

Cancelar

NUBOM — Ni Un Bocado Menos

Diplomado Desarrollo de Sistemas con Tecnología Java · DGTIC UNAM

Figura E9. Formulario de receta con ingredientes principales y complementarios

Inicio
Hogar
Recetas
Lista de compras
Reportes

Armando Luna

RECETAS

Recetario de la comunidad

Recetas compartidas por otros usuarios y aprobadas por un administrador.

+ Nueva receta

Comunidad

Mis recetas

Buscar por nombre

Arroz rojo

Por ti · 30 min

El acompañamiento de siempre, con jitomate natural.

★ 3.5 (2)

Ver receta

Ensalada de espinaca y queso

Por Ana Torres · 15 min

Fresca, rápida y usa lo que ya tienes en el refrigerador.

★ 4.5 (2)

Ver receta

Sopa de verduras

Por Ana Torres · 35 min

Ideal para aprovechar verduras próximas a caducar.

★ 5.0 (2)

Ver receta

Tostadas de pan con yogurt

Por ti · 10 min

Desayuno ligero para no desperdiciar el pan que está por vencer.

★ 4.5 (2)

Ver receta

NUBOM — Ni Un Bocado Menos

Diplomado Desarrollo de Sistemas con Tecnología Java · DGTIC UNAM

Figura E10. Recetario de la comunidad con búsqueda y promedio de calificaciones

34

[Inicio](#)
[Hogar](#)
[Recetas](#)
[Lista de compras](#)
[Reportes](#)

Armando Luna

RECETA

Ensalada de espinaca y queso

Por Ana Torres · 15 min · 4.5 (2 calificación(es))

Publicada

Fresca, rápida y usa lo que ya tienes en el refrigerador.

Preparación

- Lava y seca la espinaca.
- Corta el queso panela en cubos y el jitomate en gajos.
- Mezcla todo en un tazón.
- Adereza con aceite, limón y sal al gusto.

Ingredientes

PRINCIPALES	
Espinaca	1 paquete
Queso panela	200 g
COMPLEMENTARIOS	
Jitomate	2 pza
Limón	1 pza

Tu calificación

★★★★★

Cambiar calificación

Quitar mi calificación

[← Volver a recetas](#)

NUBOM — Ni Un Bocado Menos

Diplomado Desarrollo de Sistemas con Tecnología Java · DGTIC UNAM

Figura E11. Detalle de una receta publicada y calificación del usuario

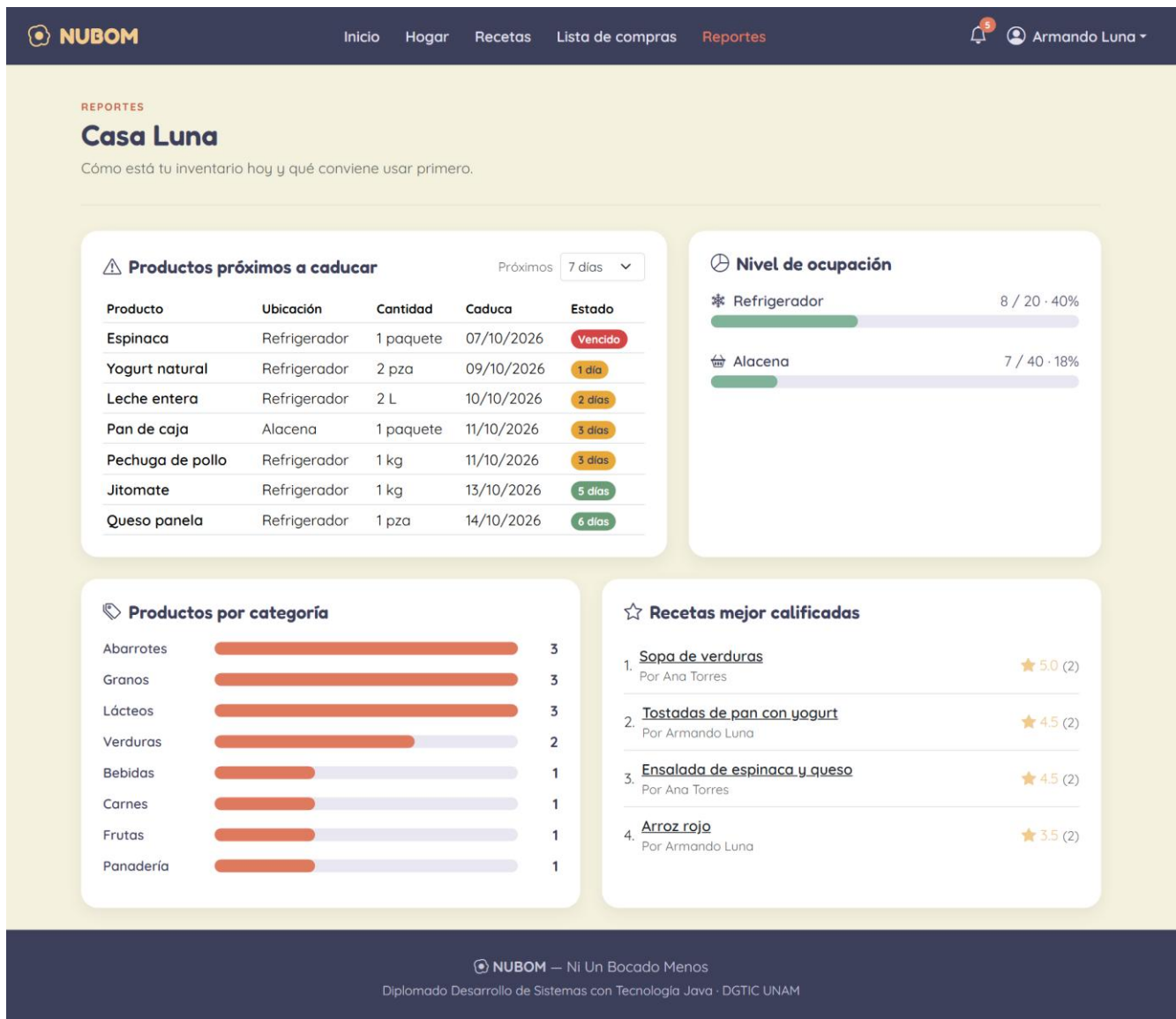


Figura E12. Los cuatro reportes: próximos a caducar, nivel de ocupación, productos por categoría y recetas mejor calificadas

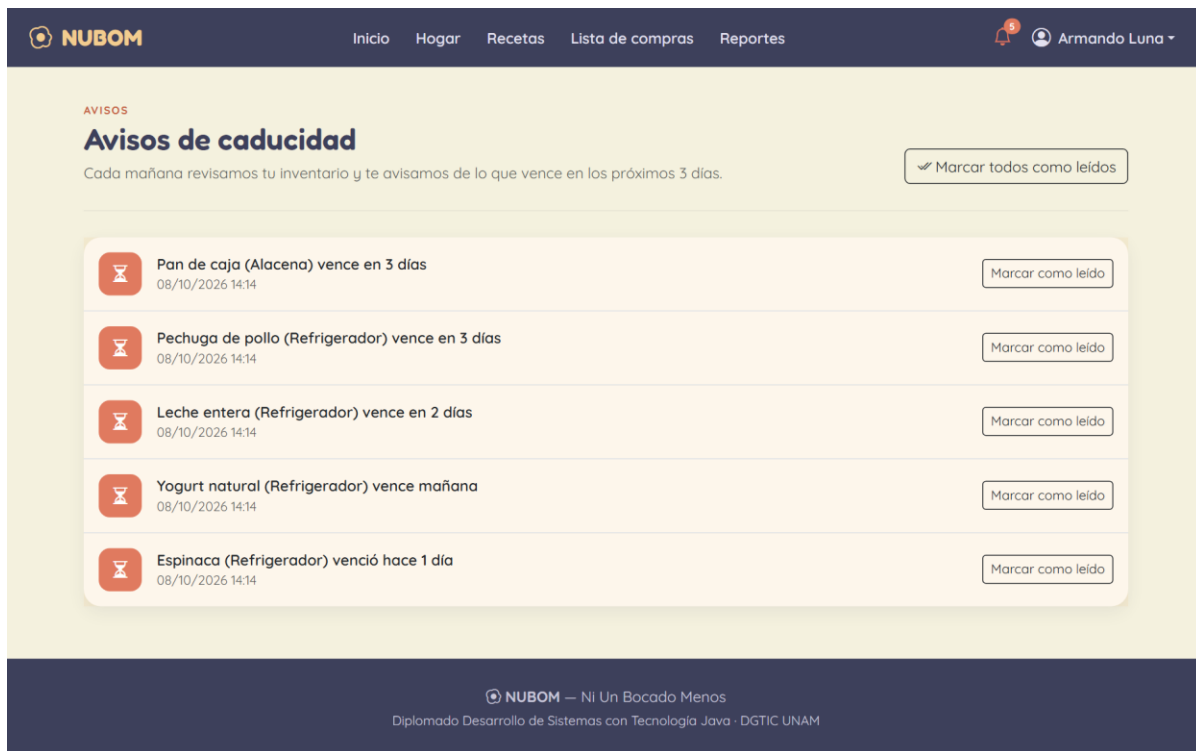


Figura E13. Avisos de caducidad generados por la tarea programada

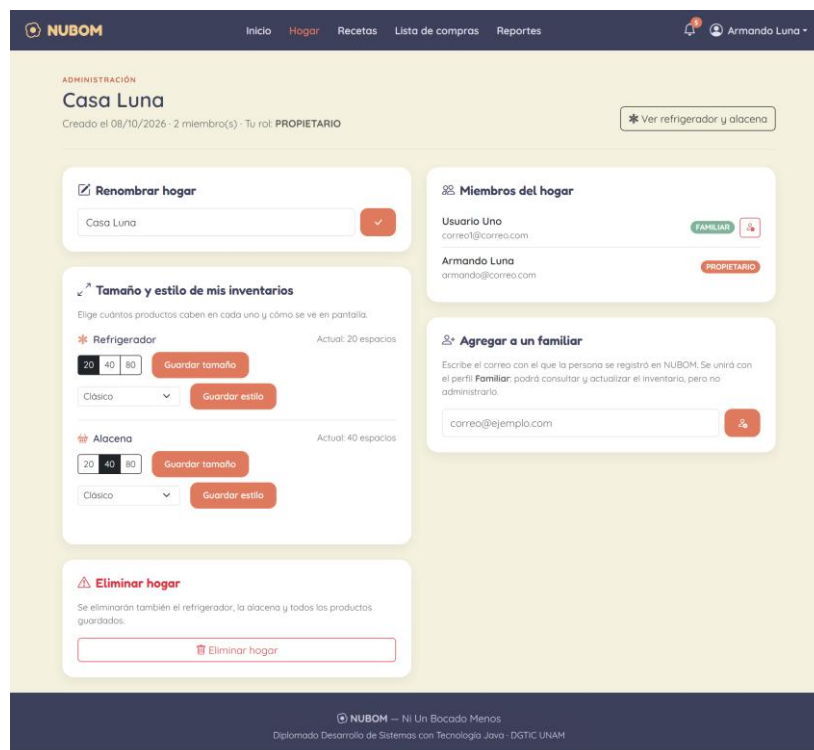


Figura E14. Administración del hogar (propietario): nombre, tamaño y estilo de inventarios, integrantes

Interfaz responsive



Figura E16. Hogar en un teléfono (390 px de ancho)

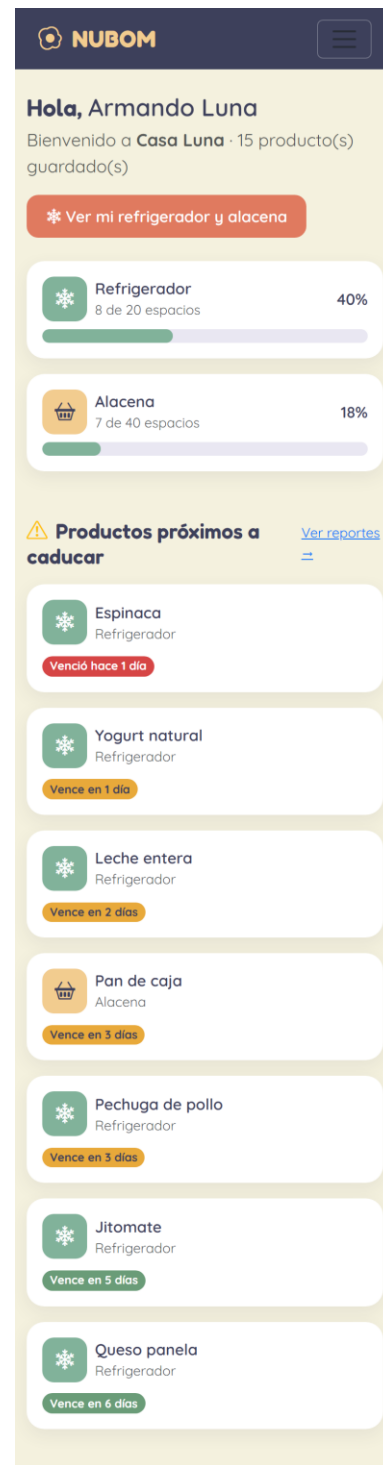


Figura E17. Inicio en un teléfono

11.2 Evidencia de seguridad

Aplicación web

• • • Aplicación web · acceso sin sesión y sin token CSRF

```
# Sin sesión, una página protegida redirige al formulario de inicio de sesión
$ curl -i http://localhost:8080/hogar
HTTP/1.1 302
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Set-Cookie: JSESSIONID=5BB7B1228C72A05FB284AC951E7BA2A3; Path=/; HttpOnly; SameSite=Lax
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Location: http://localhost:8080/login
Content-Length: 0
Date: Thu, 08 Oct 2026 20:15:42 GMT

# Un POST sin el campo _csrf es rechazado por la cadena web
$ curl -i -X POST -d "correo=armando@correo.com" -d "contrasena=..." http://localhost:8080/login
HTTP/1.1 403
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Set-Cookie: JSESSIONID=18044DF37E1FC8C06BBD55422E6C2868; Path=/; HttpOnly; SameSite=Lax
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: text/html; charset=UTF-8
Content-Language: es-MX
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:42 GMT
```

Figura E21. Sin sesión, una página protegida redirige a /login (302); un POST sin token CSRF se rechaza (403). La cookie de sesión lleva HttpOnly y SameSite=Lax

• • • MariaDB · las contraseñas se guardan como hash BCrypt

```
$ mysql -u root -p nubom -e "select id_usuario, correo, rol_sistema, concat(left(contrasena, 24), '...') as contrasena_bcrypt, char_length(contrasena) as longitud from usuario"
```

id_usuario	correo	rol_sistema	contrasena_bcrypt	longitud
1	armando@correo.com	USUARIO	\$2a\$10\$L1f20YjqyUsMiaD7N...	60
2	correo1@correo.com	USUARIO	\$2a\$10\$L1f20YjqyUsMiaD7N...	60
3	correo2@correo.com	USUARIO	\$2a\$10\$L1f20YjqyUsMiaD7N...	60
4	admin@correo.com	ADMIN	\$2a\$10\$L1f20YjqyUsMiaD7N...	60
5	ana@correo.com	USUARIO	\$2a\$10\$L1f20YjqyUsMiaD7N...	60

Figura E22. Las contraseñas se almacenan como hash BCrypt de 60 caracteres; rol global de cada usuario

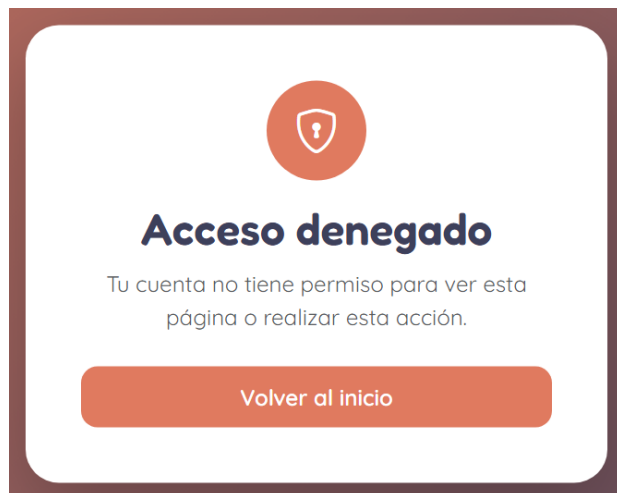


Figura E15. Un usuario sin rol de administrador intenta abrir /admin/recetas

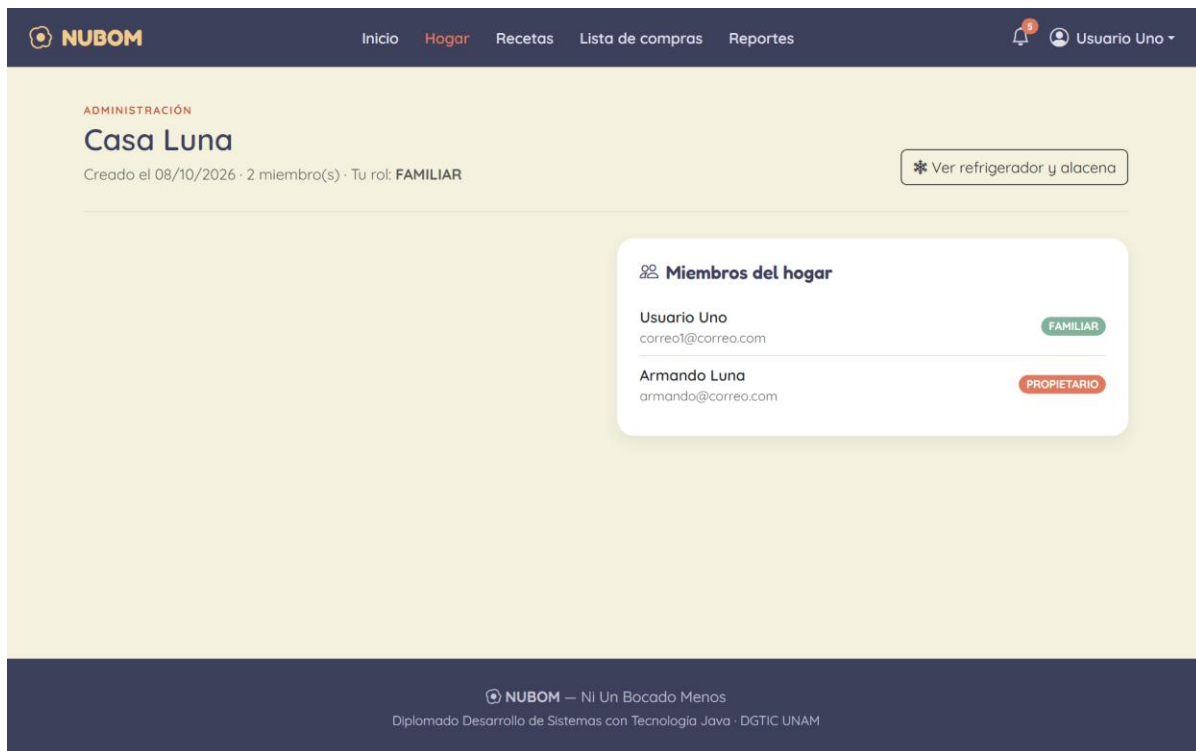


Figura E18. El perfil Familiar ve a los integrantes, pero no las opciones de administración del hogar



Figura E19. Bandeja de moderación, visible solo para el administrador (enlace «Moderación» en el menú)

• • • API REST · petición sin token (401) y con token (200)

```
# Sin token
$ curl -i http://localhost:8080/api/v1/reportes/ocupacion
HTTP/1.1 401
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json;charset=UTF-8
Content-Length: 198
Date: Thu, 08 Oct 2026 20:15:45 GMT

{"timestamp":"2026-10-08T14:15:45.8700744","status":401,"error":"Unauthorized","mensaje":"Se requiere un token de acceso válido (Authorization: Bearer <token>)","ruta":"/api/v1/reportes/ocupacion"}

# Con el access token
$ curl -i -H "Authorization: Bearer $ACCESS_TOKEN" http://localhost:8080/api/v1/reportes/ocupacion
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:45 GMT

[{"idInventario":1,"inventario":"Refrigerador","productos":8,"capacidad":20,"porcentaje":40},{"idInventario":2,"inventario":"Alacena","productos":7,"capacidad":40,"porcentaje":18}]
```

Figura E24. La misma petición sin token (401 con ErrorDetail) y con el access token (200)

• • • API REST · autorización por rol y por hogar (403)

```
# Un usuario sin rol de administrador no puede modificar catálogos
$ curl -i -X POST -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" -d "{\"nombre\":\"INVITADO\"}" http://localhost:8080/api/v1/roles

HTTP/1.1 403
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json; charset=UTF-8
Content-Length: 159
Date: Thu, 08 Oct 2026 20:15:47 GMT

{"timestamp":"2026-10-08T14:15:47.0130271","status":403,"error":"Forbidden","mensaje":"No tienes permiso para realizar esta operación","ruta":"/api/v1/roles"}

# Tampoco puede consultar un hogar al que no pertenece
$ curl -i -H "Authorization: Bearer $ACCESS_TOKEN" http://localhost:8080/api/v1/hogares/3

HTTP/1.1 403
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:47 GMT

{"timestamp":"2026-10-08T14:15:47.0465917","status":403,"error":"Forbidden","mensaje":"No tienes permiso para realizar esta operación","ruta":"/api/v1/hogares/3"}
```

Figura E25. Autorización: 403 al modificar un catálogo sin ser administrador y al consultar un hogar ajeno

- • • API REST · renovación con rotación del refresh token

```
# Primera renovación: entrega un par nuevo y revoca el refresh token usado
$ curl -i -X POST -H "Content-Type: application/json" -d @refresh.json http://localhost:8080/api/v1/auth/refresh

HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:48 GMT

{"tokenType": "Bearer", "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbmRvQGNCbnJlby5jb20iLCJ1aWQiOiJEsInjVbCI6IlVTVFUSSU8iLCJpYXQiOiJE3OTE0OTAwNDgsImV4cCI6MTc5MTQ5MTQ0OH0.bJaX2tM2gzhv0rciwqLKQ0Sba7iiGmLc9Grmu7bGiwm", "expiresIn": 900, "refreshToken": "YnWXxeUjVjcUmItEUBSLcwr347eYZbstcTS_O_JjK7I"}

# El mismo refresh token ya no sirve: se detecta la reutilización
$ curl -i -X POST -H "Content-Type: application/json" -d @refresh.json http://localhost:8080/api/v1/auth/refresh

HTTP/1.1 401
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:48 GMT

{"timestamp": "2026-10-08T14:15:48.1197193", "status": 401, "error": "Unauthorized", "mensaje": "El refresh token ya fue utilizado; se cerraron todas las sesiones de la API", "ruta": "/api/v1/auth/refresh"}
```

Figura E26. Rotación del refresh token: la primera renovación entrega un par nuevo; reutilizar el anterior responde 401

• • • API REST · listado paginado y validación

```
$ curl -i -H "Authorization: Bearer $ACCESS_TOKEN" "http://localhost:8080/api/v1/productos?size=2&sort=fechaCaducidad,asc"

HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:49 GMT

{"contenido":[{"idProducto":14,"nombre":"Aceite vegetal","cantidad":1,"unidad":"L","categoria":"Abarrotes","fechaIngreso":"2026-10-08T14:14:52","activo":true,"idInventario":2,"fechaCaducidad":null,"codigoBarras":null,"estadoCaducidad":"SIN_FECHA"},{"idProducto":1,"nombre":"Espinaca","cantidad":1,"unidad":"paquete","categoria":"Verduras","fechaIngreso":"2026-10-08T14:14:52","activo":true,"idInventario":1,"fechaCaducidad":"2026-10-07","codigoBarras":null,"estadoCaducidad":"VENCIDO"}],"pagina":0,"tamano":2,"totalElementos":15,"totalPaginas":8}

# Datos inválidos: 400 con el detalle por campo
$ curl -i -X POST -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" -d '{"idInventario":1,"nombre":"","cantidad":0}' "http://localhost:8080/api/v1/productos"

HTTP/1.1 400
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 08 Oct 2026 20:15:49 GMT
Connection: close

{"timestamp":"2026-10-08T14:15:49.3219322","status":400,"error":"Bad Request","mensaje":"Los datos enviados no son válidos","ruta":"/api/v1/productos","detalles":[{"unidad": "Selecciona una unidad","cantidad": "La cantidad debe ser al menos 1","nombre": "El nombre del producto es obligatorio","categoria": "Selecciona una categoría"}]}
```

Figura E27. Listado paginado de productos y error de validación con detalle por campo

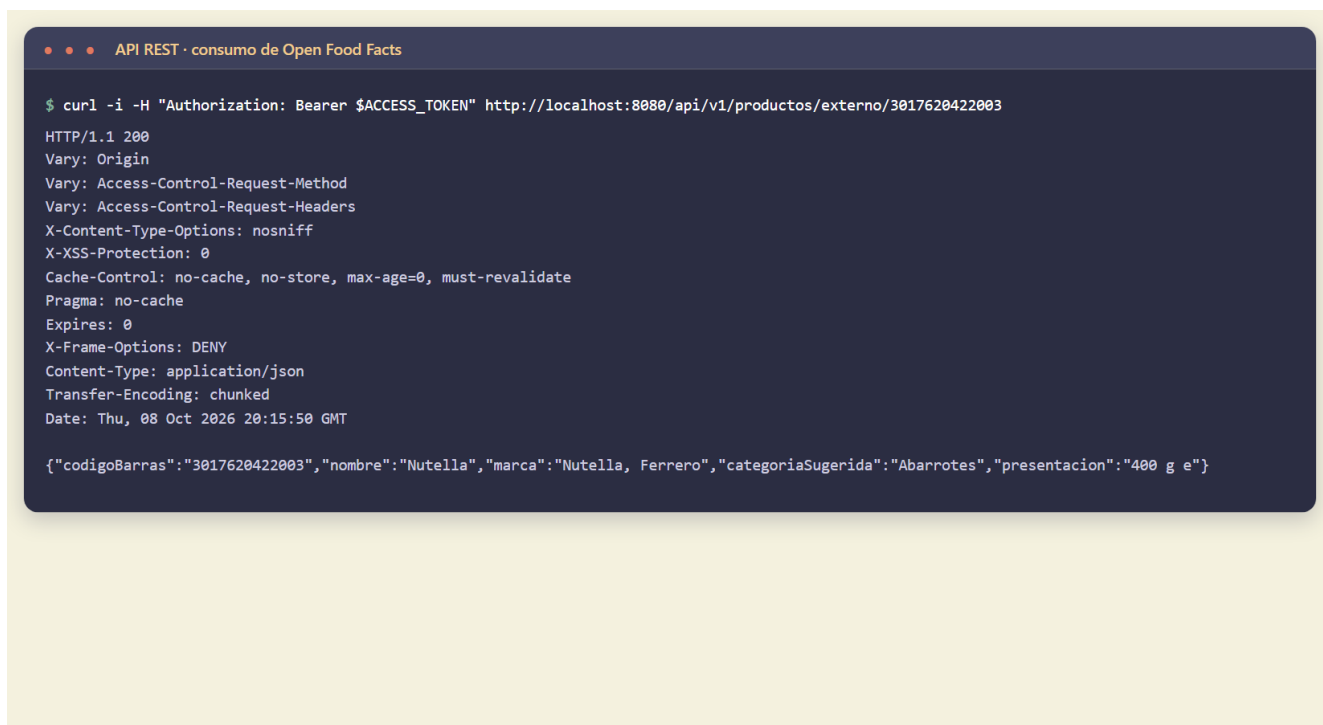


Figura E28. Consumo de la API externa Open Food Facts a través de NUBOM

11.3 Pruebas automatizadas

Resultado de `mvnw.cmd test`: 27 pruebas, 0 fallas.

Clase de prueba	Tipo	Pruebas	Qué verifica
<code>JwtServiceTest</code>	Unitaria	4	Emisión del JWT y rechazo de tokens expirados, con otra firma o mal formados.
<code>RefreshTokenServiceTest</code>	Unitaria (Mockito)	5	Solo se guarda el hash; rotación; la reutilización revoca todas las sesiones; expiración.
<code>ProductoServiceImplTest</code>	Unitaria (Mockito)	6	Capacidad máxima, fecha de caducidad, pertenencia al hogar, baja lógica y semáforo de caducidad.
<code>SeguridadIntegracionTest</code>	Integración (MockMvc)	11	Las dos cadenas: redirección al login, formulario, CSRF, 401 sin token, 403 por rol y por hogar, refresh de un solo uso.
<code>SpringWebApplicationTests</code>	Integración	1	El contexto de Spring arranca completo.

12. Código fuente

12.1 Repositorio

Elemento	Ubicación
Repositorio Git	https://github.com/ArmandoLunas/nubom
Proyecto	<code>spring-web/</code> , artefacto Maven <code>dgtic.core:spring-web</code>
Esquema de base de datos	<code>esquema-base-datos.sql</code> (referencia) y <code>src/main/resources/data.sql</code> (datos iniciales)

12.2 Documentación en Markdown

El archivo `README.md`, en la raíz del proyecto, explica para qué sirve el sistema, qué se necesita para correrlo, las variables de entorno, los pasos de ejecución, los usuarios de prueba y un ejemplo de uso de la API con tokens. En la carpeta `docs/` están este documento, las capturas de evidencia, la documentación de la API del módulo 7 y la guía de pruebas con curl.

12.3 Instrucciones de despliegue

Requisitos

- JDK 17.
- MariaDB 10.7 o posterior en ejecución en el puerto 3306.
- Conexión a internet para Open Food Facts y, si se desea probar el correo, una cuenta SMTP.

Pasos

1. Opcionalmente, definir las variables de entorno. En desarrollo local no hacen falta: hay valores por omisión. La base de datos nubom se crea sola en el primer arranque.

```
set NUBOM_DB_USER=root
set NUBOM_DB_PASSWORD=<contraseña de MariaDB>
set NUBOM_JWT_SECRET=<clave aleatoria de al menos 32 bytes en Base64>
rem Solo para enviar correos reales:
set NUBOM_MAIL_ENABLED=true
set NUBOM_MAIL_USER=<cuenta de correo>
set NUBOM_MAIL_PASSWORD=<contraseña de aplicación>
```

2. Ejecutar la aplicación desde la raíz del proyecto.

```
mvnw.cmd spring-boot:run
```

3. Abrir `http://localhost:8080` e iniciar sesión con alguno de los usuarios de prueba definidos en `data.sql` y listados en `README.md` (propietario, familiar, usuario sin hogar y administrador).
4. Para la API, obtener un token en POST `http://localhost:8080/api/v1/auth/login` y enviarlo como Bearer en las demás peticiones.

En un ambiente que no sea el de desarrollo hay que definir `NUBOM_DB_PASSWORD` y `NUBOM_JWT_SECRET`, y cambiar `spring.jpa.hibernate.ddl-auto` a `validate` y `spring.sql.init.mode` a `never`, para que el esquema y los datos no se vuelvan a generar en cada arranque.

Empaquetado y pruebas

```
mvnw.cmd test
mvnw.cmd clean package
java -jar target\spring-web-0.0.1-SNAPSHOT.jar
```

13. Conclusiones

El cambio más grande de esta entrega fue la seguridad. En las entregas anteriores el acceso se resolvía con un interceptor que revisaba un atributo de la sesión, y las contraseñas se comparaban en texto plano. El reto fue pasar todo eso a Spring Security sin romper lo que ya funcionaba, porque había usuarios de prueba, formularios y redirecciones que dependían de la forma anterior. Al terminar me quedó claro que la seguridad conviene dejársela al framework. Quité código que yo había escrito y a cambio obtuve cosas en las que ni había pensado, como el cifrado de contraseñas, la protección CSRF, el cambio del identificador de sesión al entrar y un solo lugar donde viven las reglas de acceso.

Otro problema fue hacer convivir la interfaz web y la API dentro del mismo proyecto, cuidando que el manejo de errores de una no se metiera con la otra. Una petición a la API sin permiso tiene que recibir un 401 en JSON y no una redirección al formulario de inicio de sesión. De ahí aprendí que un mismo sistema puede necesitar dos formas de autenticación. La sesión con CSRF es lo natural para páginas que se generan en el servidor y el JWT lo es para una API sin estado, y separarlas en dos cadenas de filtros fue lo que resolvió el problema. Dentro de ese mismo tema, lo que más vueltas me hizo dar fue el refresh token, porque había que decidir qué se guarda, cómo se revoca y qué hacer si alguien presenta un token que ya se había usado.

También me costó distinguir el rol global del usuario (usuario o administrador) del rol que tiene dentro de un hogar (propietario o familiar). Este último depende del hogar, así que no puede ser una autoridad fija. Con eso entendí en la práctica que autenticar no es lo mismo que autorizar. No basta con saber quién es el usuario, en cada operación hay que revisar que el recurso sea de su hogar. Fue la parte más laboriosa y también la más fácil de olvidar.

En la parte de datos, pasar de guardar las llaves foráneas como enteros a usar asociaciones de JPA simplificó mucho las consultas, aunque me obligó a tener cuidado con la carga perezosa y a no convertir entidades directamente a JSON. Los DTOs resultaron ser la solución a eso, y además me dejaron agregar columnas y relaciones sin cambiar las respuestas de la API que ya estaban documentadas. Si empezara otra vez, modelaría las relaciones con JPA desde el primer día.

Por último, depender de Open Food Facts me hizo pensar en qué pasa cuando un servicio externo está lento o caído. El alta de productos no podía quedarse esperando, así que la consulta tiene tiempos de espera cortos y, si falla, el formulario simplemente se llena a mano.

Mejoras futuras

- Motor de sugerencia de recetas según el porcentaje de ingredientes disponibles en el inventario.
- Generación de recetas y categorización automática de productos con inteligencia artificial.
- Escaneo de productos y de tickets de compra con la cámara.
- Invitación a un hogar mediante enlace enviado por correo, con aceptación explícita (hoy el propietario incorpora directamente a una persona ya registrada).
- Inicio de sesión con proveedores externos mediante OAuth2.
- Migraciones de esquema versionadas con Flyway y despliegue en contenedores.
- Documentación interactiva de la API con OpenAPI.